Laboratory 3
Supply Chain model

This laboratory shows how to model a simple supply chain. The supply chain consists of one distribution center and several retailers spread across the country. The retailers order product delivery and the distributor sends trucks with the product to them.

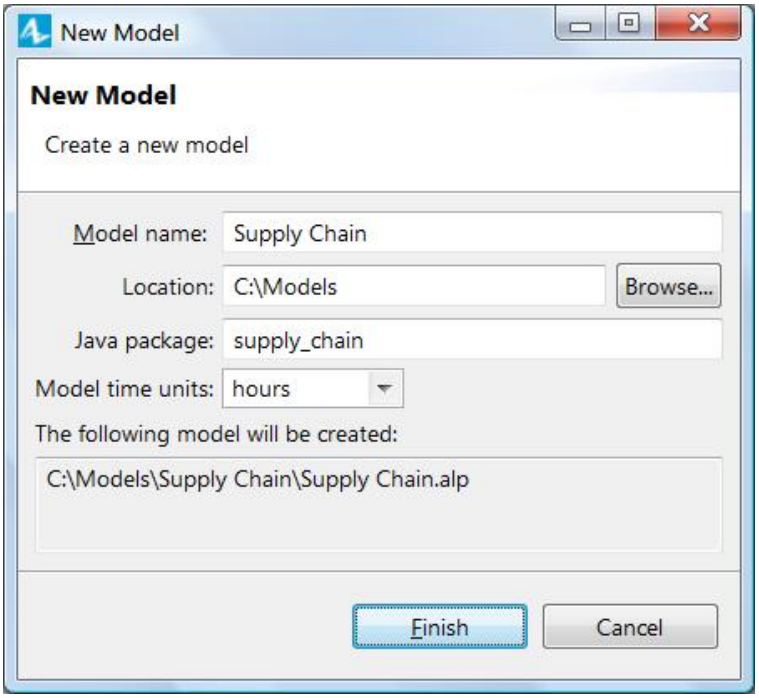
We will position the agents on the world map in this model. When a retailer requests product, the distribution center starts processing it, and sends a truck to deliver it. Then this truck should return to its base location. Trucks move along the roads on the map.

Phase 1. Configuring the GIS Map

In this phase, we will create a model, then add and configure our first component, a tiled GIS (Geographic Information System) map. We will navigate the map both at the design-time and at runtime.

1. Create a new model by selecting **File** > **New** > **Model** from AnyLogic's main menu. The **New Model** wizard will open.

2. Specify the name of the model. Type *Supply Chain* in the **Model name** edit box.



3. Specify the location where you want to store your model files. Browse for the existing folder using the **Browse** button, or type the name of the folder you want to create in the **Location** edit box.

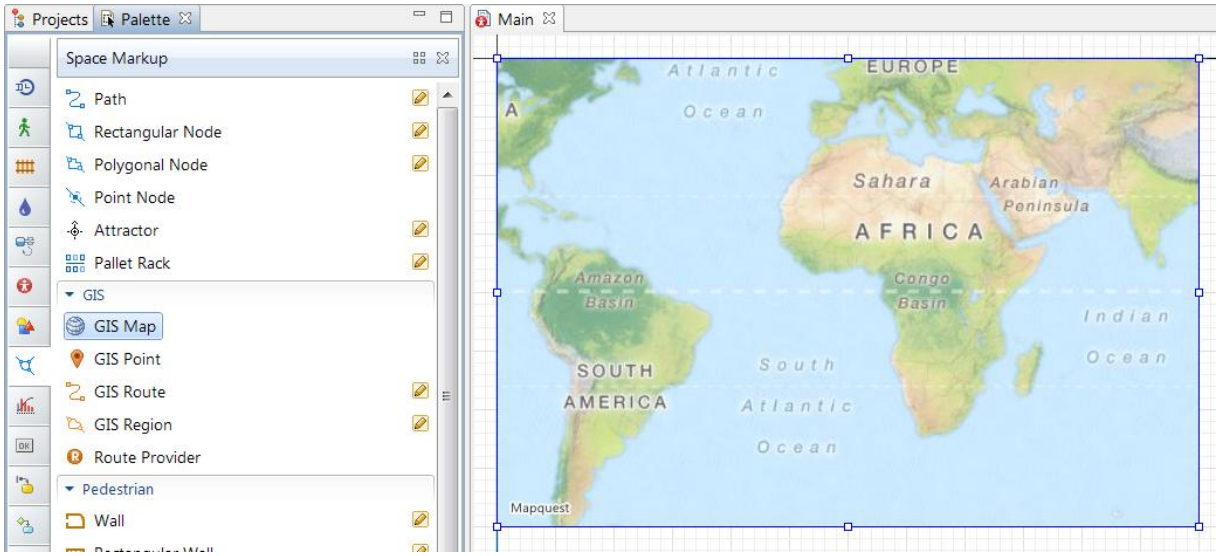
4. Select *hours* as the **Model time units**.

5. Click **Finish** to complete the process.

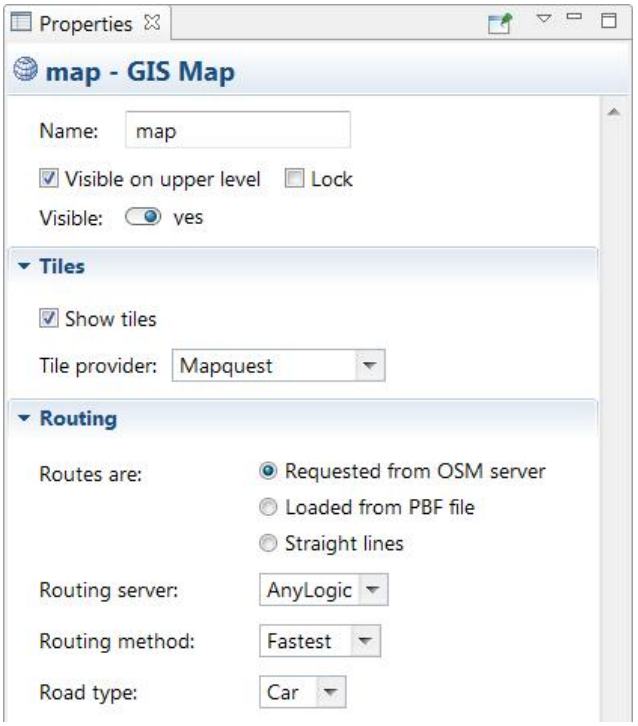
   We will start with adding a GIS map onto the *Main* agent type diagram. All agents, that we will create later, will live on *Main*, and that means in the GIS space.

6. The **Palette** view opens by default the **Process Modeling Library** palette, and we need an element from a different palette. Hover the mouse over the vertical navigation bar, wait for the list of palettes to expand, and choose the **Space Markup** palette.

7. In the **Space Markup** palette, find the section **GIS**, and drag the **GIS Map** shape from the palette onto the graphical editor.



When you first drop the GIS map onto the agent's diagram, it displays the world map using tiles. We will use the default configuration of the **GIS Map** shape in AnyLogic:



8. Let us change the map size: it is common to have the map fill all presentation window. Drag one of rectangular handles to resize the shape in the graphical editor. The presentation window border is marked on this agent's diagram with the blue frame.

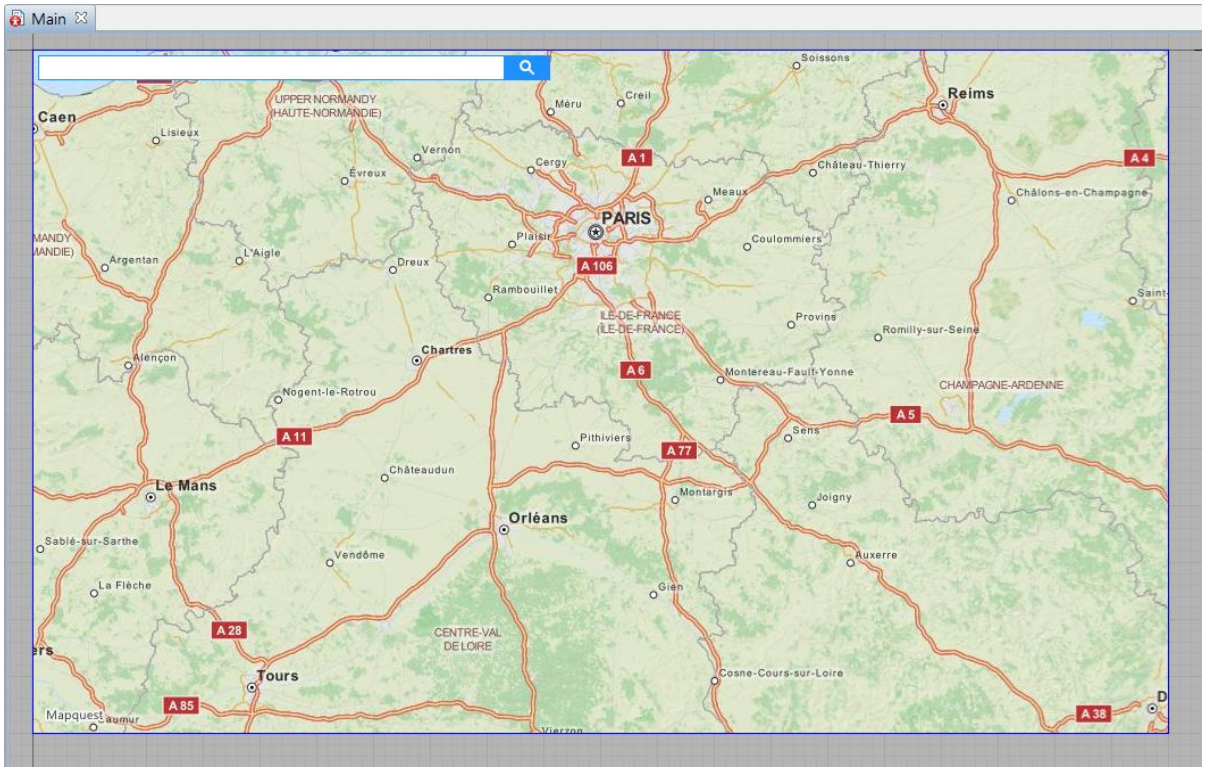
   Another popular technique is to turn off panning of the presentation window itself, leaving only the map panning at the runtime.

   Next, let us choose the area that we want to use in the model and display it on the map.

9. Double-click the map, or right-click it and select **Edit map** from the context menu. You will enter the map edit mode, the map will remain highlighted unlike the rest of the graphical editor. Now you can navigate the world map to select some specific area:

   - To pan the map, hold down the left mouse button as you move it.
   - To zoom, move your mouse wheel towards or from you.

10. Choose some area, for instance, a country with several big cities between which you would like to create a supply chain system. We have chosen *France* here.

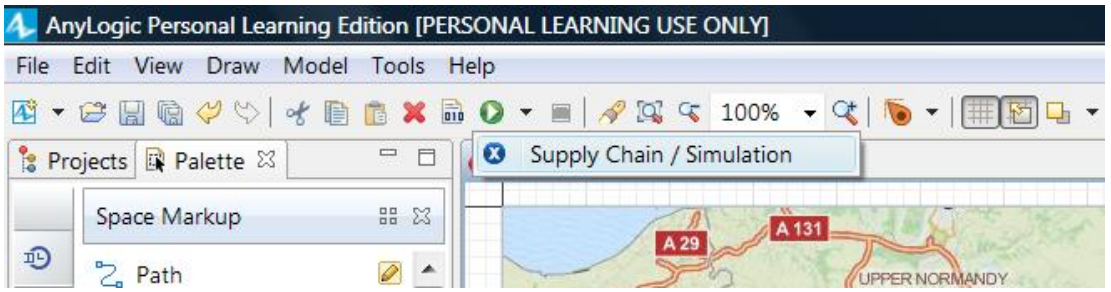


11. To exit the map edit mode, click the grayed out area of the graphical editor outside the map, or right-click the map and select the option **Finish map editing** from the context menu.

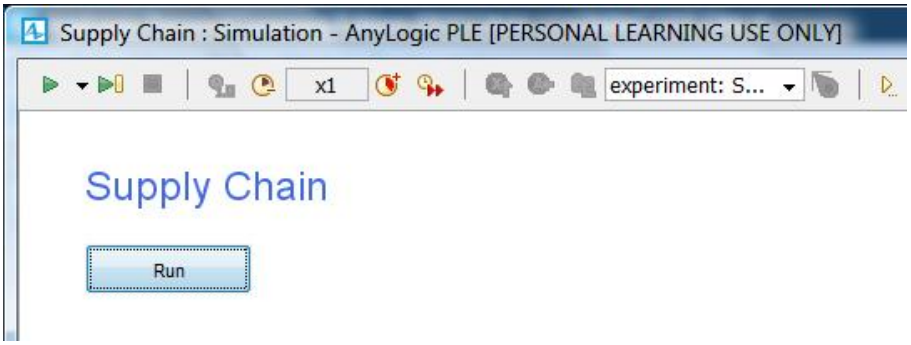
    We have finished setting up the **GIS Map** shape on the agent's diagram. Let us run the model and navigate the map at runtime.

12. Build your project by clicking the **Build Model** toolbar button. If there are some errors in your model, the building fails and the **Problems** view appears listing all the errors found in your model. Double-click an error in the list to open the location of the error and fix it. After the model is successfully built, you can start it. Running the simulation, you automatically bring the current model up to date.

13. Choose the experiment you want to run from the drop-down list of the **Run** toolbar button. Your simulation experiment is called **Supply Chain GIS / Simulation**. Later on you can use the **Run** toolbar button to start the previously run experiment.

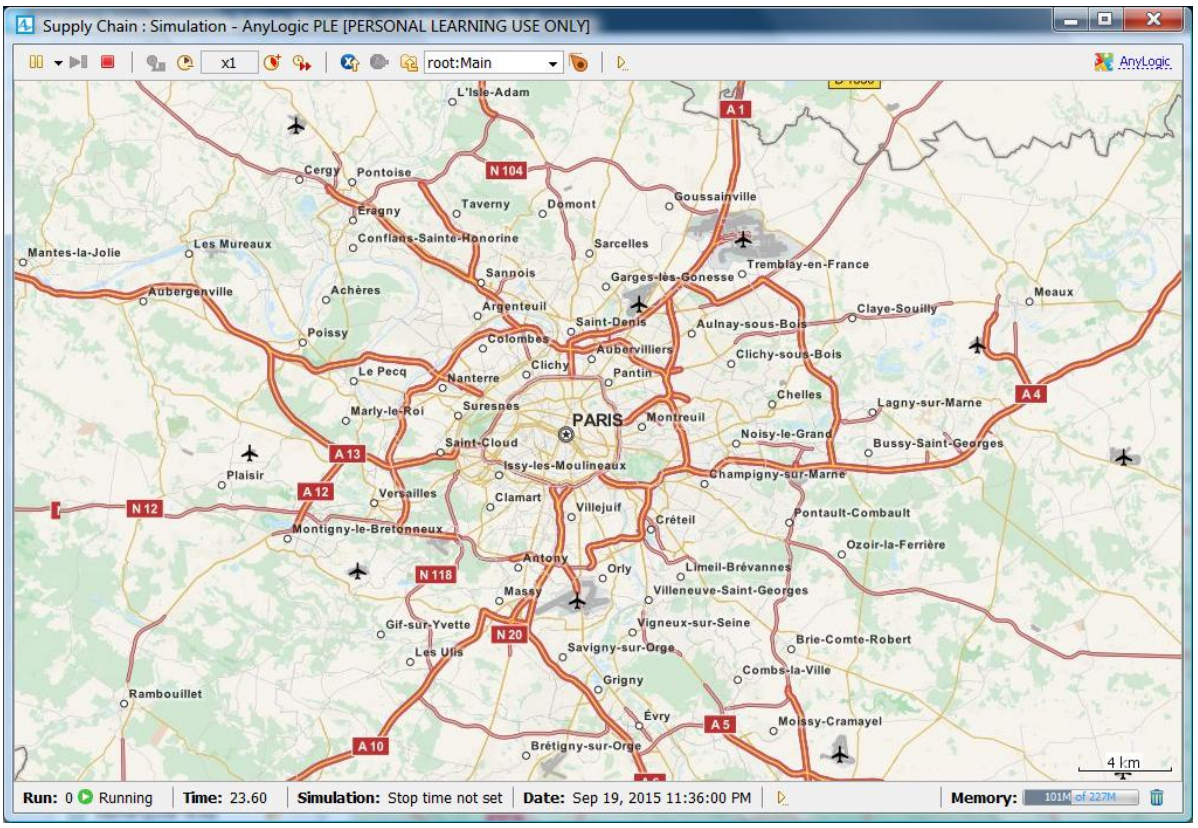


14. Having started the model, you will see the presentation window. It displays the presentation designed for your simulation experiment. AnyLogic automatically places the show-bench title and the button enabling running your model and switching to the presentation designed for the *Main* agent type.



15. Click the **Run** button. The model will run and display the map. You can navigate the GIS map at runtime using the same commands as at the design-time. We only started developing the model, so there is only the map now. But as we run the model, and navigate the map, the tiles get cached in the model folder. Next time, the model with the map will be running faster, using this cache.

16. You can check if AnyLogic requests tiles or routes at runtime of the GIS model: the network indicator will be active. You can find it in the status bar of the presentation window:



Next, we will create several agents and place them on the map.

Phase 2. Creating Agents

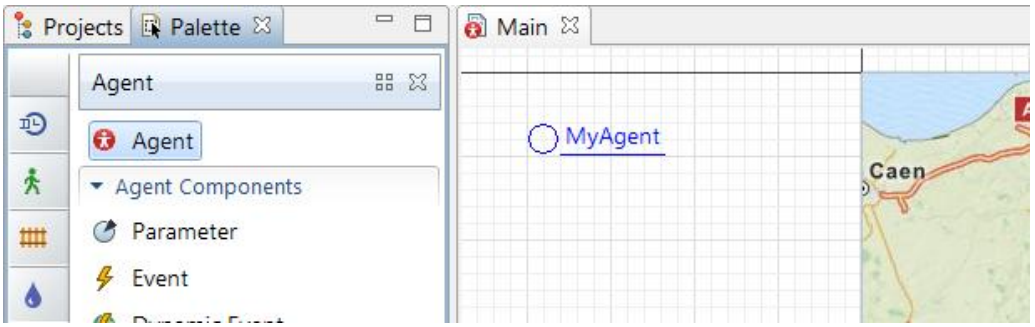
The supply chain that we model includes one distribution center, several retailers spread across some country, and a fleet of trucks that deliver the product from the distributor to retailers.

You have three choices when you create an agent: a population, a single agent, and an agent type. Each choice suggests that you create an agent type anyway, but a population or a single agent also represent the agent instances that are placed in some environment, the *Main* agent in our case. Unlike a single agent, a population is a collection of a number of indexed agents.

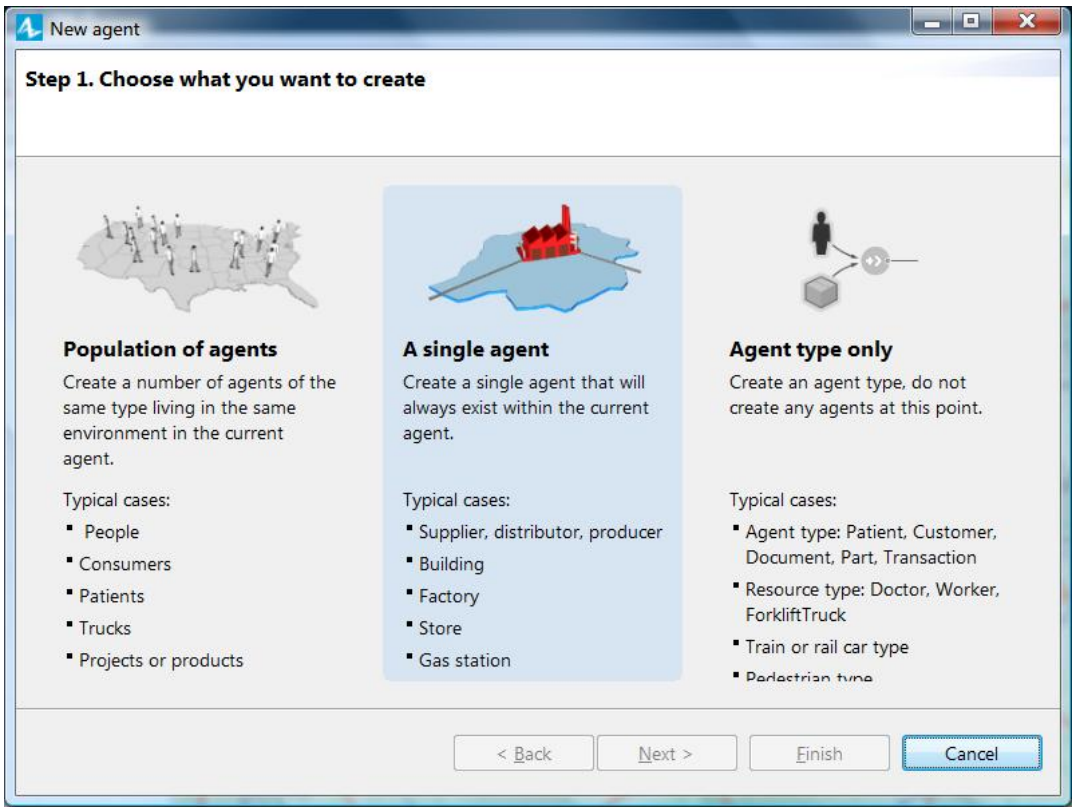
To make an agent type play the role of environment for other agents, you create those agents on its diagram.

Since we model only one distribution center, let us create it as a single agent.

1. Open the **Agent** palette in the **Palette** view. Drag the **Agent** element onto the *Main* diagram. The **New agent** wizard will pop up.

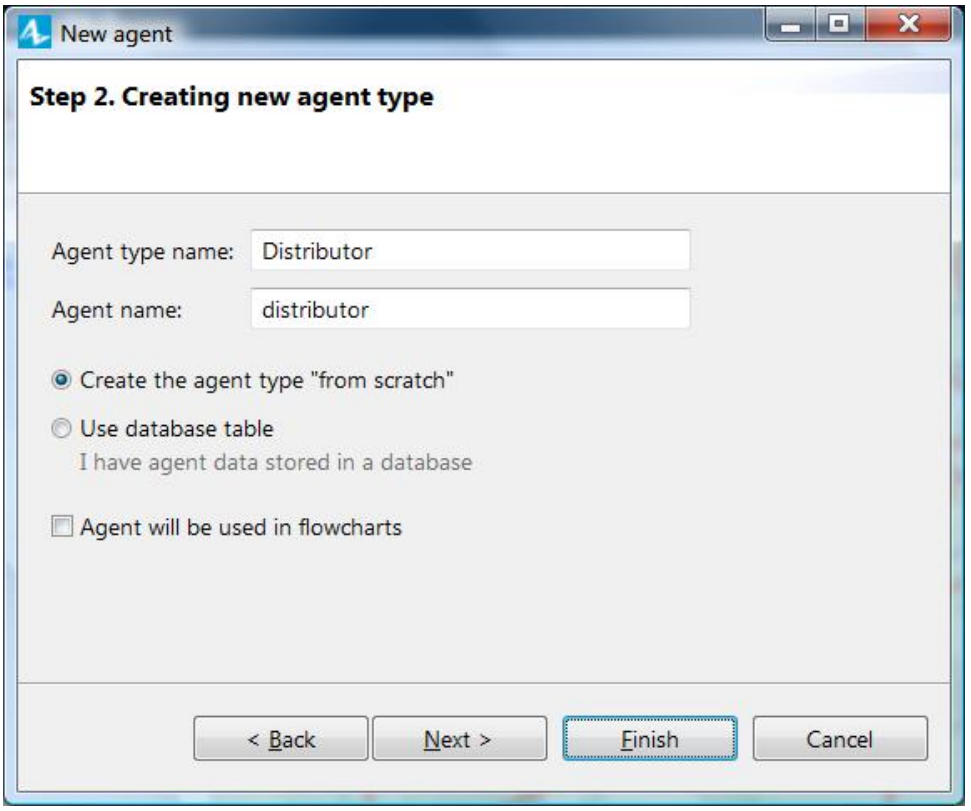


2. Choose to create **A single agent**. The wizard will take you to the next step.



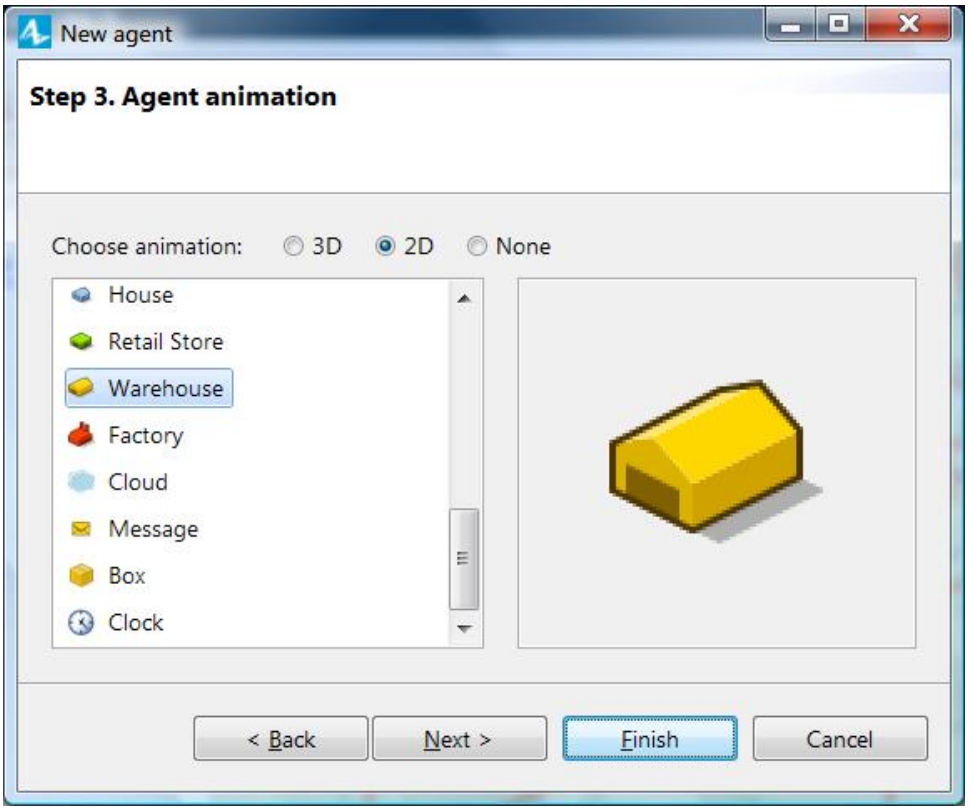
3. We are creating all our agents as new agents from scratch, and do not need to use any templates here.

   Define the **Agent type name**: *Distributor*. The agent's name of this type will autofill with the same name: *distributor*. It is a convenient way to name an agent. Click **Next**.



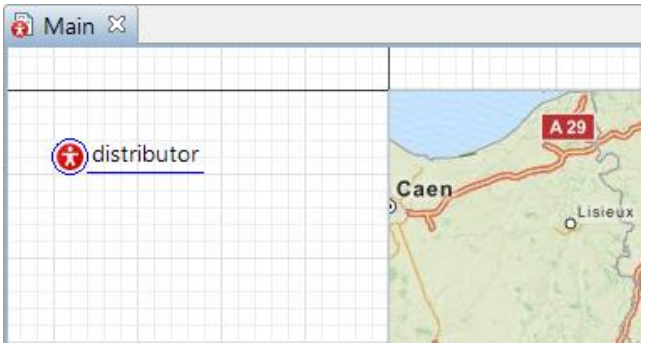
4. Next step is selecting **Agent animation**. There are choices of animation type where 3D

animation is the default one. Since we build our model on the map that is 2D, switch to the **2D** animation figures and select the **Warehouse** figure. Click **Finish**.



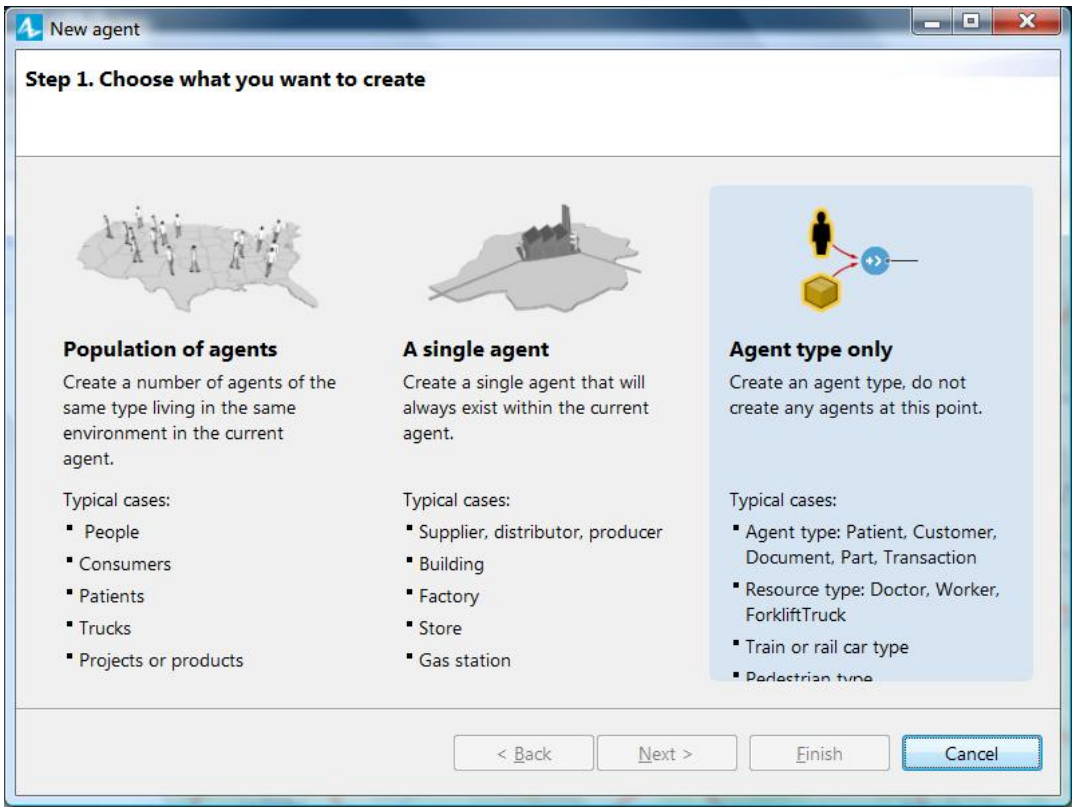
5. Next step is the **Agent parameters** page. We do not need to create here any parameters for the *Distributor* agent type so we can just skip it. Ordinarily you could choose space for the agent, but in our case the space is already defined with the map: agents that live on *Main* live in the GIS space.

   The agent object will appear on the *Main* diagram there where we have dropped it from the palette. The agent animation figure will be placed in the center of the map (it is the default location for animation on the map, and we will define a specific location for it in the next phase).

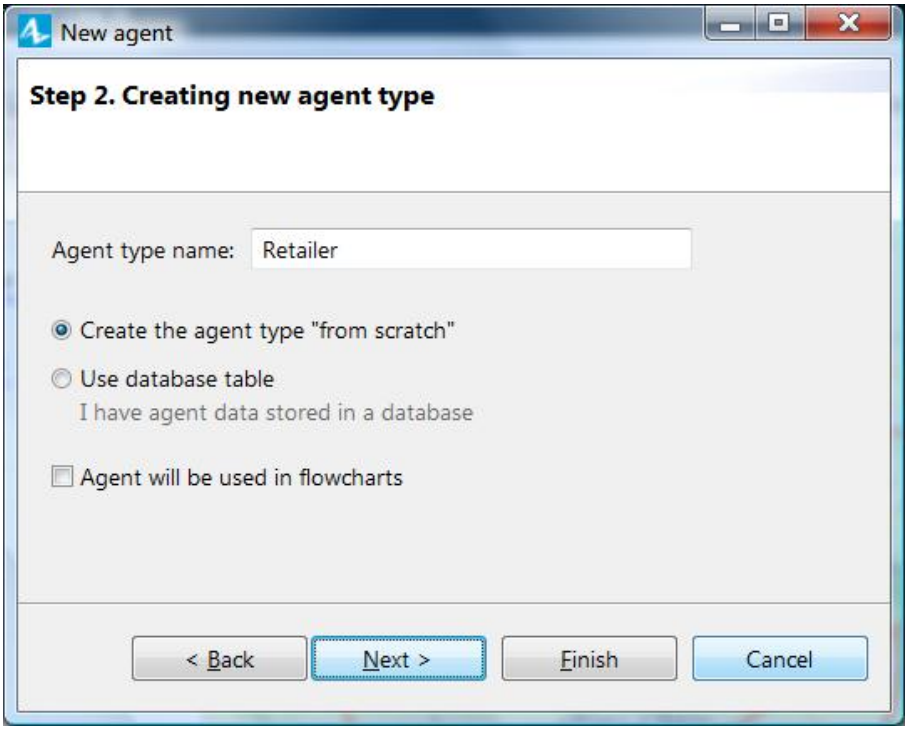


   Now we have the distributor, and let us proceed to retailers. This is a GIS model, and since GIS modeling in AnyLogic provides special means to create separate agents, we will now create just an agent type.

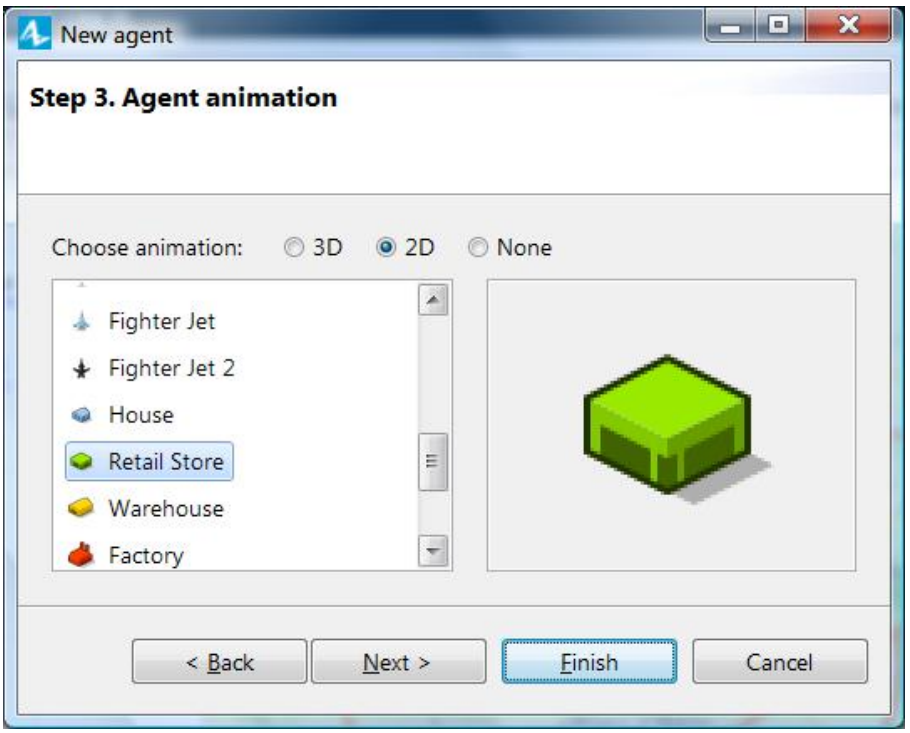
6. Drag the **Agent** element onto the *Main* diagram. Select **Agent type only** in the **New agent** wizard.



7. Specify the **Agent type name**: *Retailer*. Click **Next**.



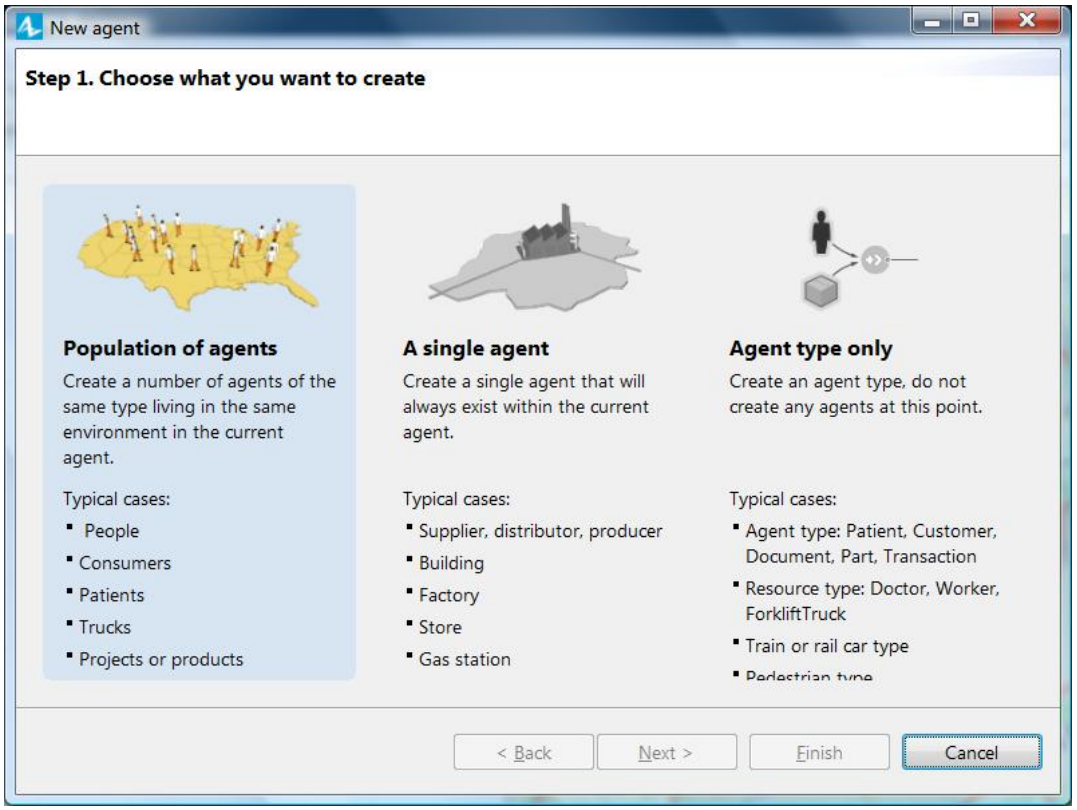
8. Again, select a **2D** animation figure to represent retailers on the map: **Retail Store**. Click **Finish** on this step. There are no parameters we need to create for this agent type.



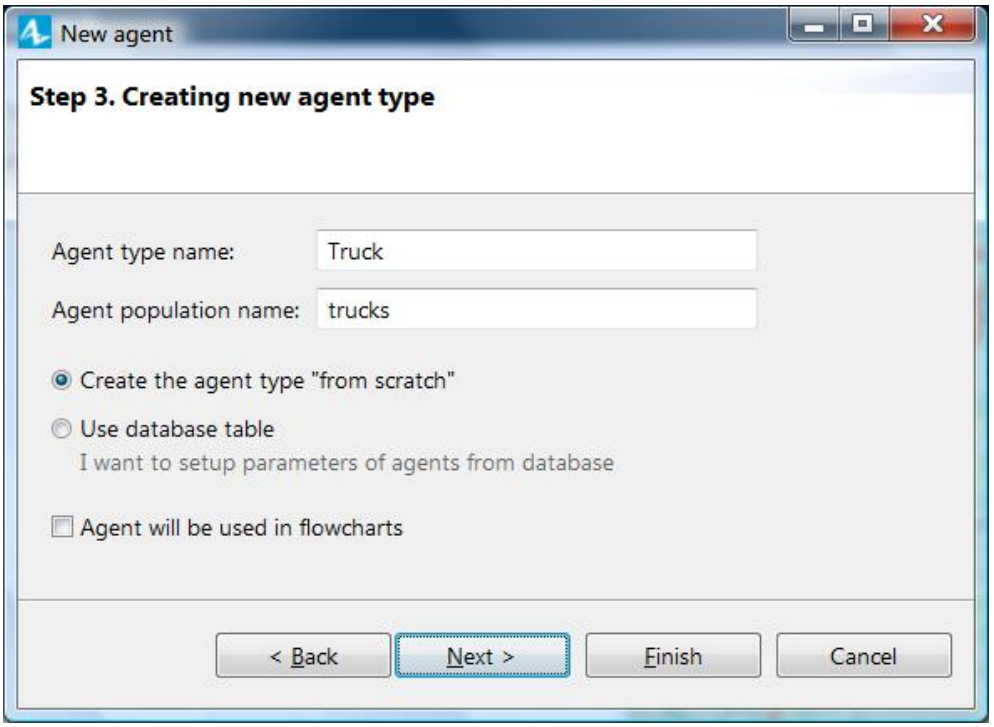
9. After you finish creating this agent type, its diagram will open automatically. The only element we have yet added onto it is the animation figure. Switch back to the *Main* diagram.

   Next, there are trucks that move between the supply chain elements. Let us create the whole fleet at once as a population.

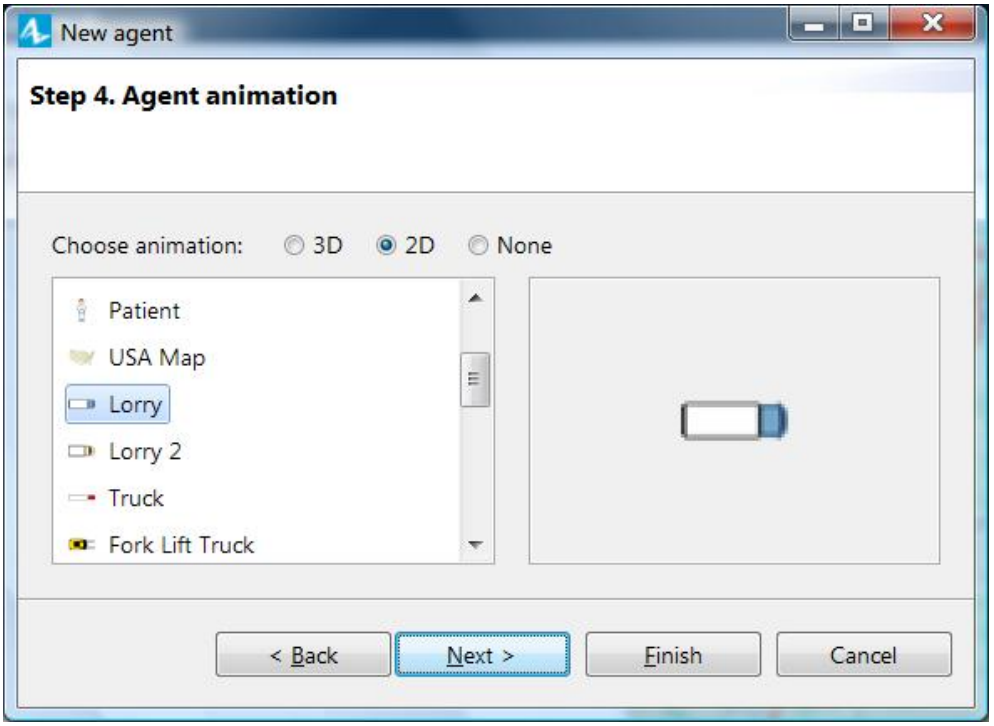
10. Drag the **Agent** element onto the *Main* diagram. This time, choose to create **Population of agents** in the first step.



11. Click **Next** in the second step, we are creating a new type of agent for this population. In the third step, define the **Agent type name**: *Truck*, and the name of population will autofill as *trucks*. Let us use this name for the population. Click **Next** to proceed to animation.

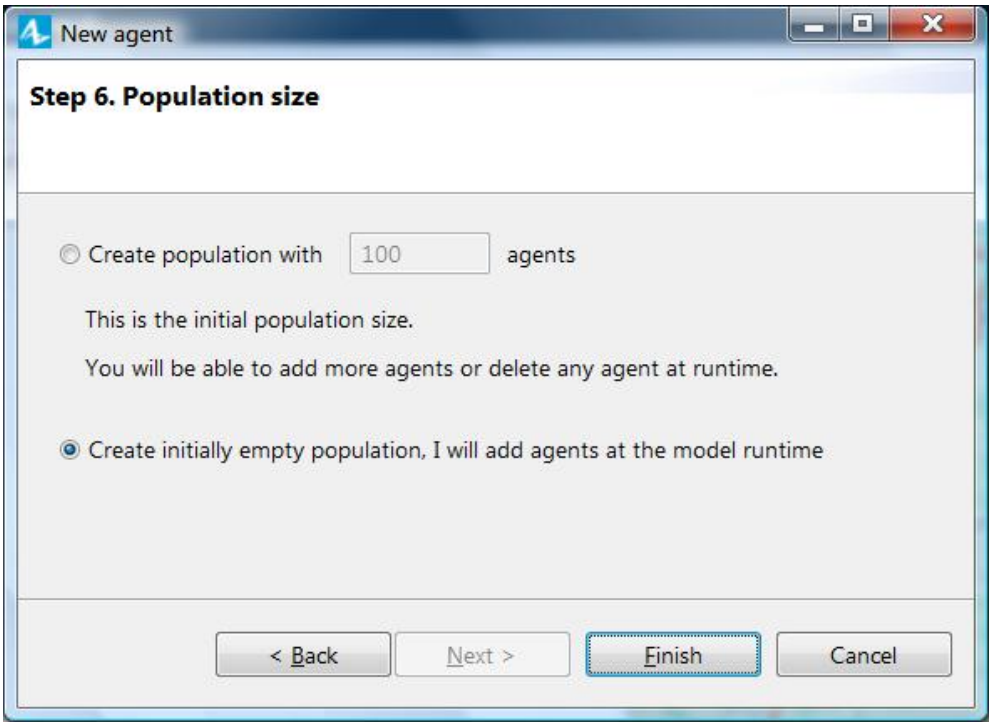


12. Here you choose the animation figure, like you did for other agents. Select a **2D Lorry** figure, for instance, or a **Truck**, and click **Next**.



13. We will skip the **Agent parameters** step again, but we need one step after that, called **Population size**. This step is peculiar to populations only (a single agent is just one agent, and an agent type does not have immediate instances).

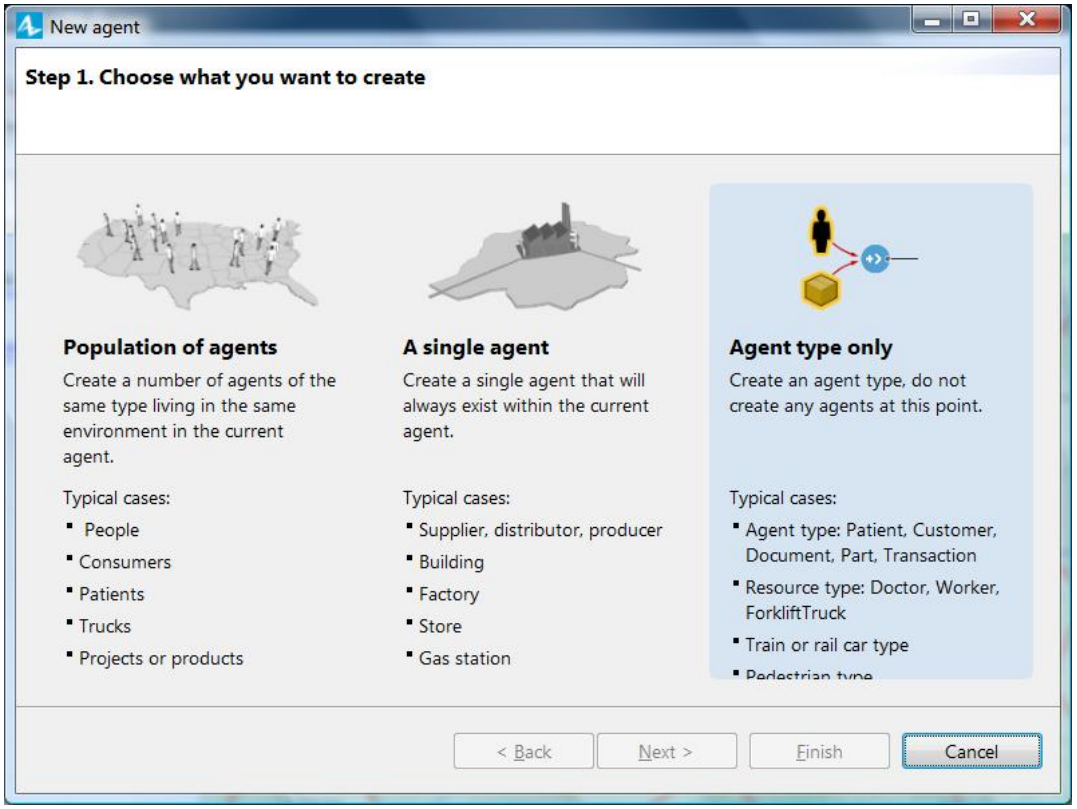
Choose to **Create initially empty population**. We will add agents later, with the help of the **Process Modeling Library** functionality.



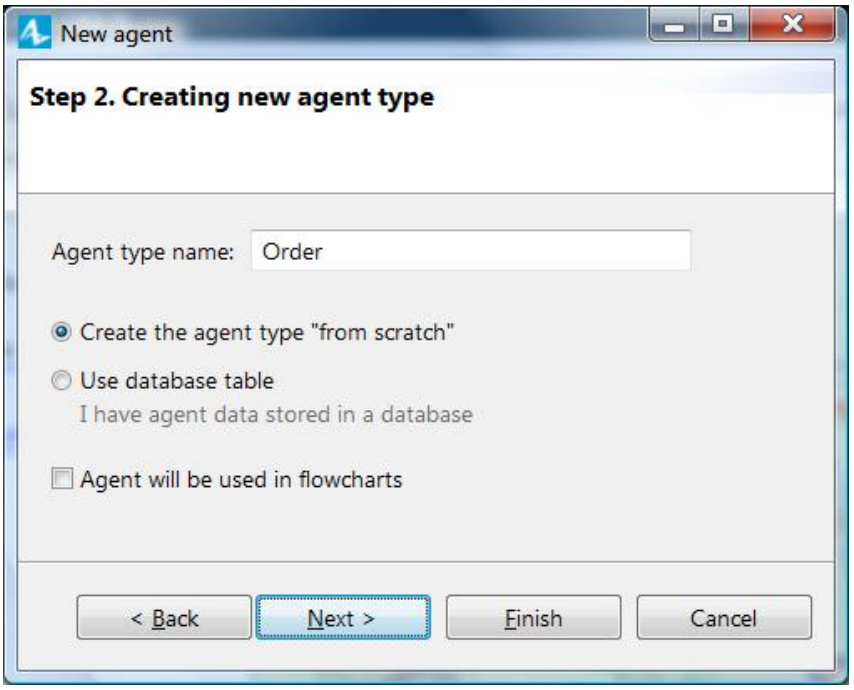
14. Click **Finish**. You will see the population *trucks[../]* residing on *Main* with its animation on the map.

    In our model, the delivery process starts when one of the retailers sends a request for the product to be delivered. We will also model the notion of order as an agent.

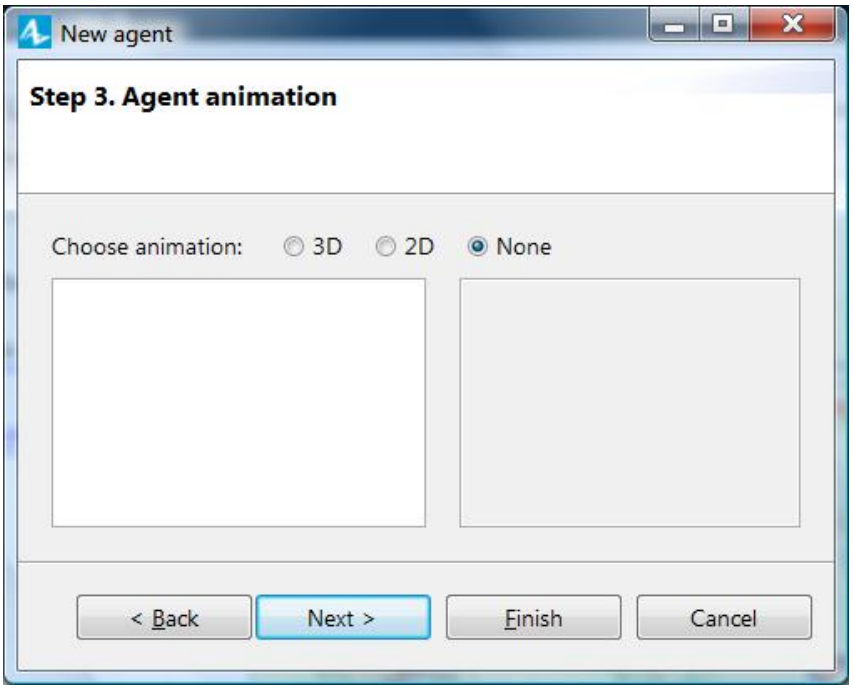
15. Drag the **Agent** element onto the *Main* diagram. Select **Agent type only** in the **New agent** wizard.



16. Define the **Agent type name**: *Order*, and click **Next**.



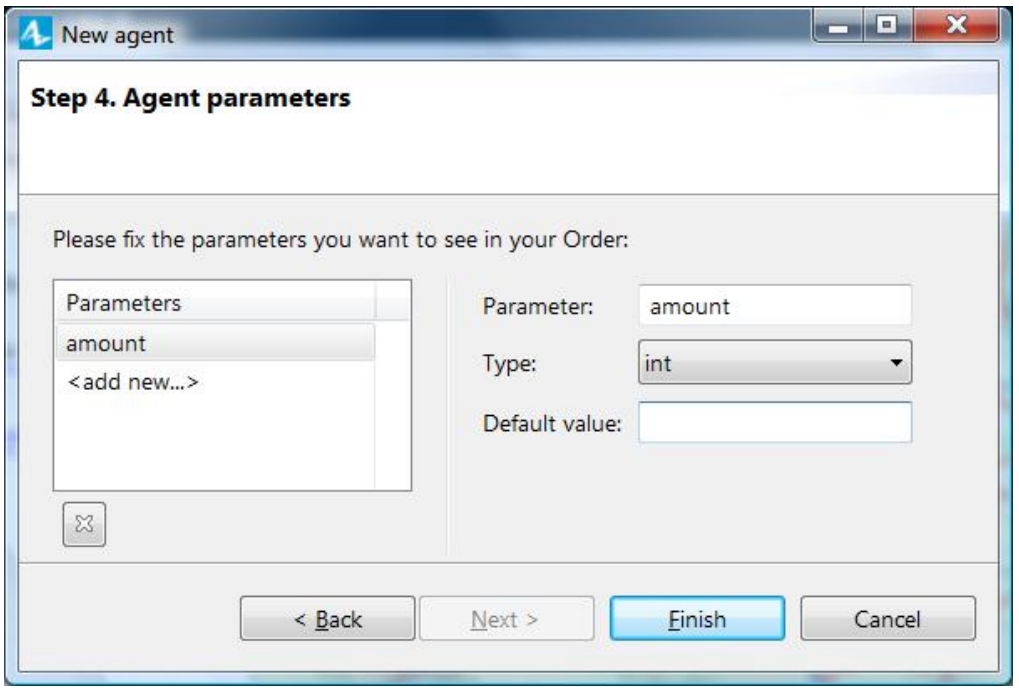
17. More often, we need to create agents to represent things that have animation, but this time choose animation: **None**. Click **Next**.



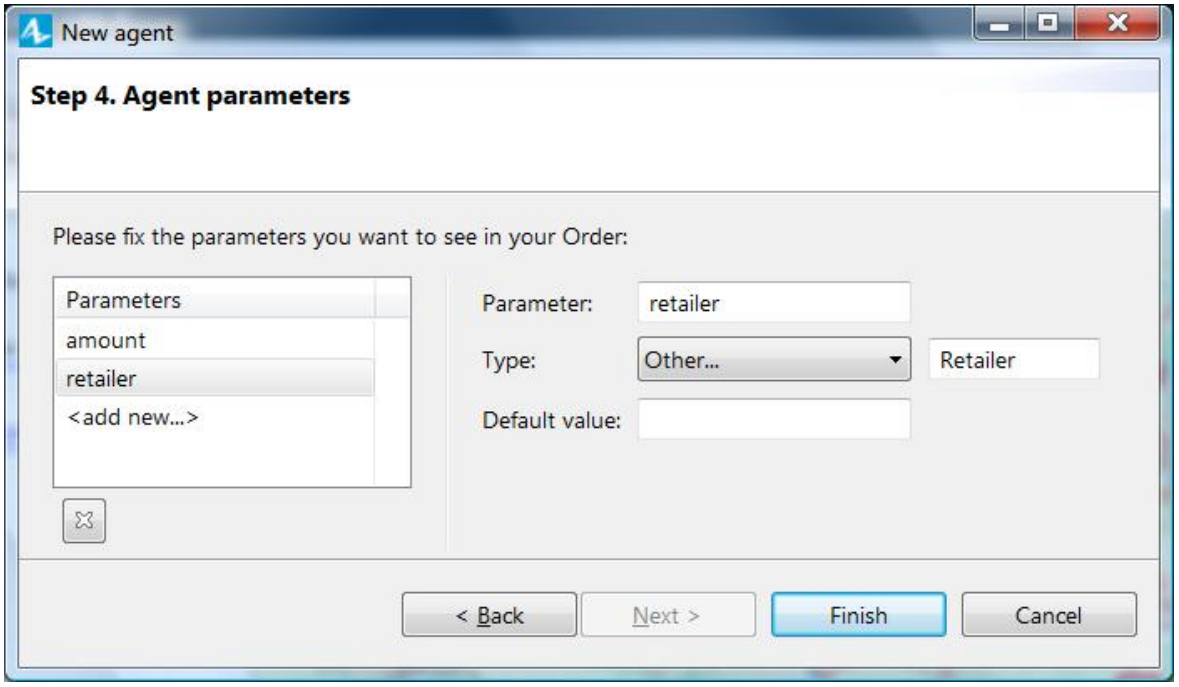
18. This time we will use the **New agent** wizard to create parameters for the agent. On the left, you can see the **Parameters** section. Click <**add new...**> there to add a parameter. The parameters' properties will appear on the right side of the wizard page.

    Specify the name in the **Parameter** field: *amount*, and then choose the **Type** below: *int* (an integer number).

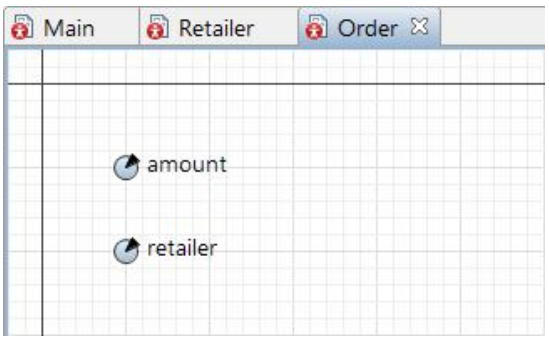
    We will use this parameter when we define how the orders are generated: the amount of product a retailer demands from the distributor.



19. Again, click **<add new...>** in the **Parameters** section. This time, we will create a parameter that defines a retailer, so let us name it *retailer* and define its type correspondingly: *Retailer*. This is a custom agent type in our model, and to use it as a parameter type, we first choose **Other...** in the **Type** drop-down list, then we specify *Retailer* in the edit box manually.



20. Click **Finish**. The *Order* agent diagram will open, and we can see what we have created in the wizard.



Go back to the *Main* diagram where we will continue developing our model. If you open the **Projects** view now you will see the agent types are the second level under the model itself. Some agents live on the *Main* diagram, i.e. *Main* plays the role of environment for those agents. You can open any agent type from the model tree to add elements on its diagram the same way we have been adding them on *Main*.

If you run the model now, you will see the distributor on the GIS map in its default location. The trucks population is still empty and we have not yet created retailers.

We will place the agents in particular locations on the map and create retailers in the next phase.

Phase 3. Placing agents in GIS space

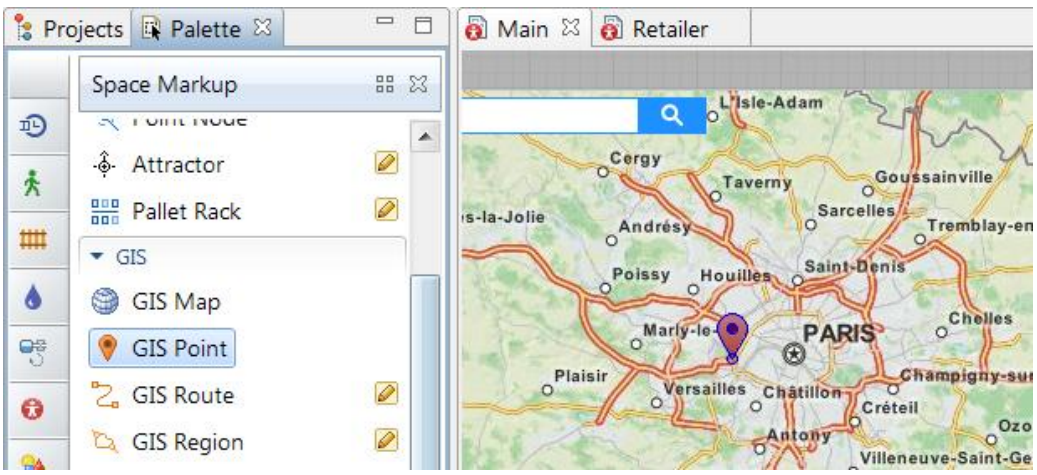
There are several different methods you can use to define an agent's position on the map, starting with specifying the geographic coordinates of some point on the map and ending with calling GIS specific methods.

We will use GIS space markup objects and the GIS search results menu. These techniques allow you to define positions on the map in a couple of mouse clicks, and do not require writing any Java code.

Since we have one distribution center, it makes sense to place it in some point on the map.

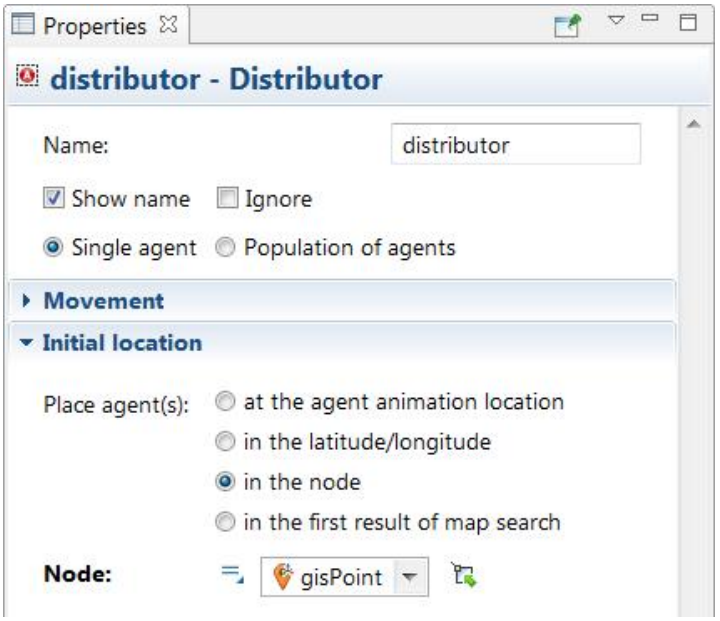
1. Choose a location on the map where you would like to place the distribution center. You can zoom the map to a smaller scale where you can even pick a certain address in some city. We chose a location somewhere near Paris.

2. Open the **Space Markup** palette and drop the **GIS Point** on the chosen location on the map.



3. Select the *distributor* agent on the *Main* and go to its **Properties** view.

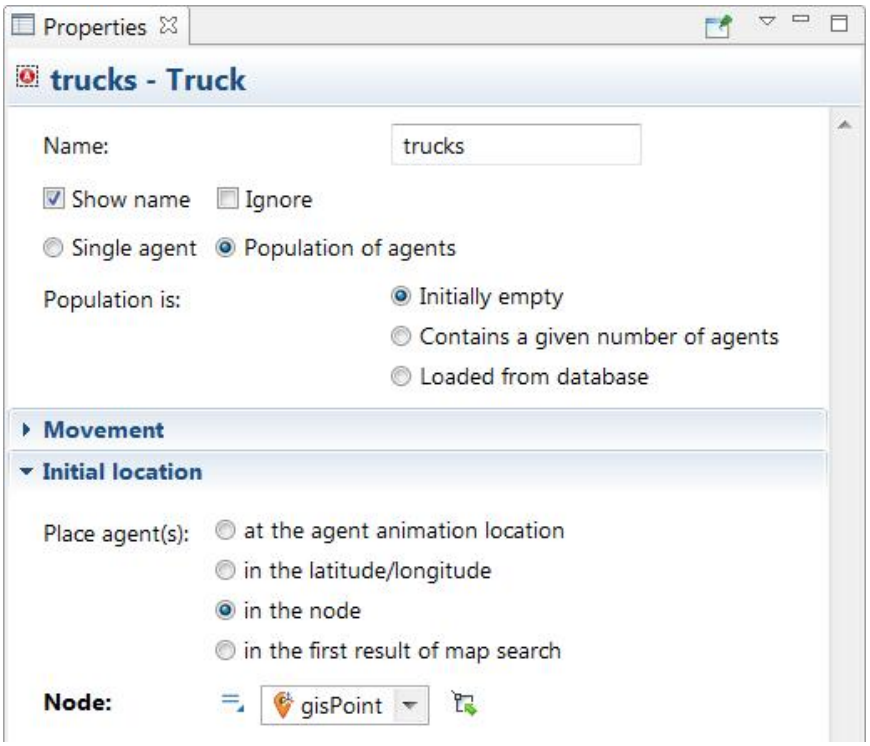
4. In the **Initial location** section, select to **Place agent(s)**: *in the node*. Then we need to specify the node. You can select the GIS point that you have created earlier from the drop-down list next to the parameter, or you can select it on the map after clicking the selection button.



5. You can run the model now and check right away the location of the distribution center animation.

   In our model, a truck delivers the product after a retailer orders it, which means it makes sense to keep the fleet at the distribution center to be able to send a loaded truck anytime in different directions.

6. Select the *trucks[../]* population and choose the same node as for the *distributor* as the trucks' initial location.



Now we will place retailers on the map while, at the same time, creating each of them as a single agent.

7. Double-click the map to enter the edit mode. The search edit box will appear. We will use it to find locations on the map and place agents there.

8. If needed, first pan and zoom the map to display several cities around your distribution center location (its GIS point on the map).

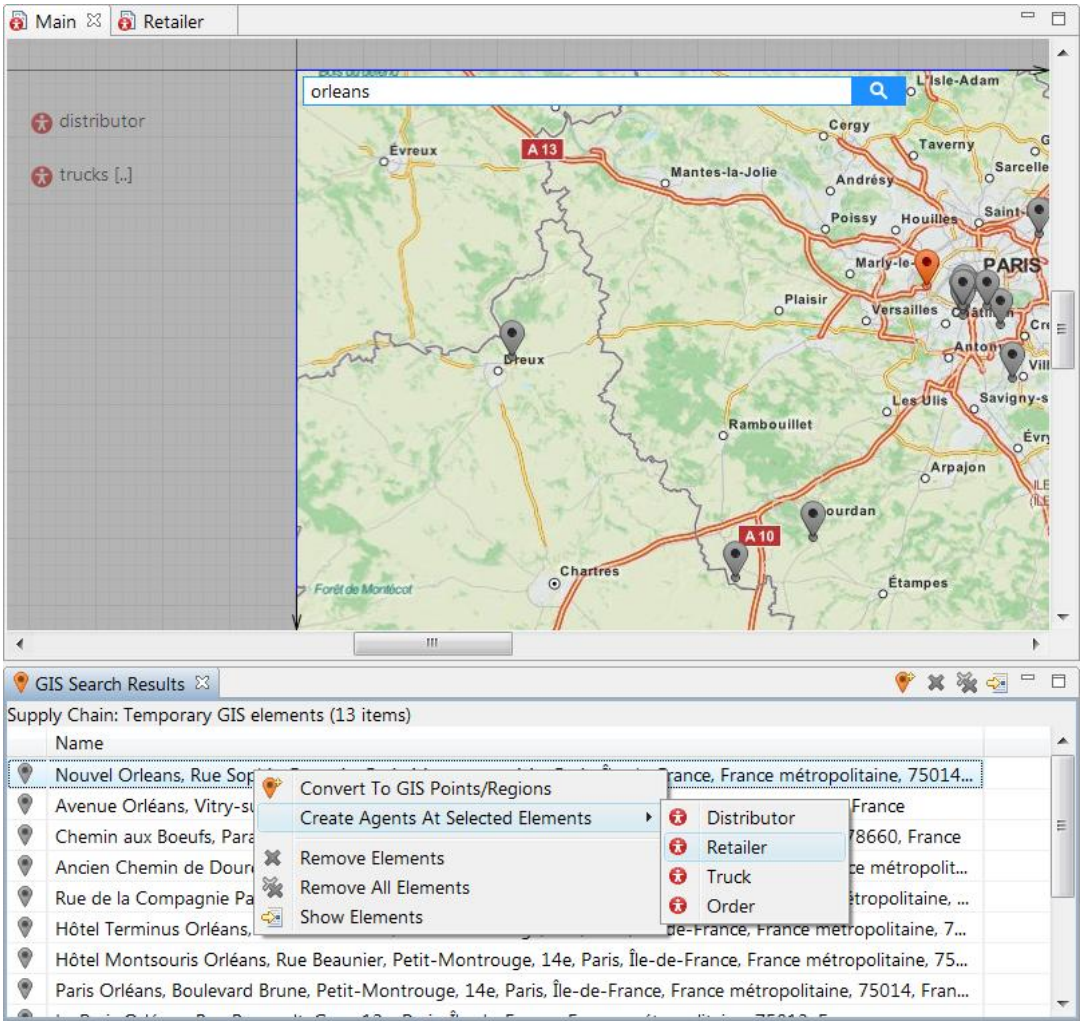
9. Type the name of the city where you want to place a retailer in the search field. Click the button  or press **Enter** on the keyboard to start searching for the location.

   The GIS search engine first checks the visible area of the world map, then, in case nothing is found, it starts searching world-wide.

10. AnyLogic will display the list of search results in the **GIS Search Results** view that opens automatically. Select one of the results as the location for a retailer. Usually, it is the first search result that you were looking for, but if you need to double-check the actual search result location, double-click it in the list, AnyLogic will highlight it on the map.

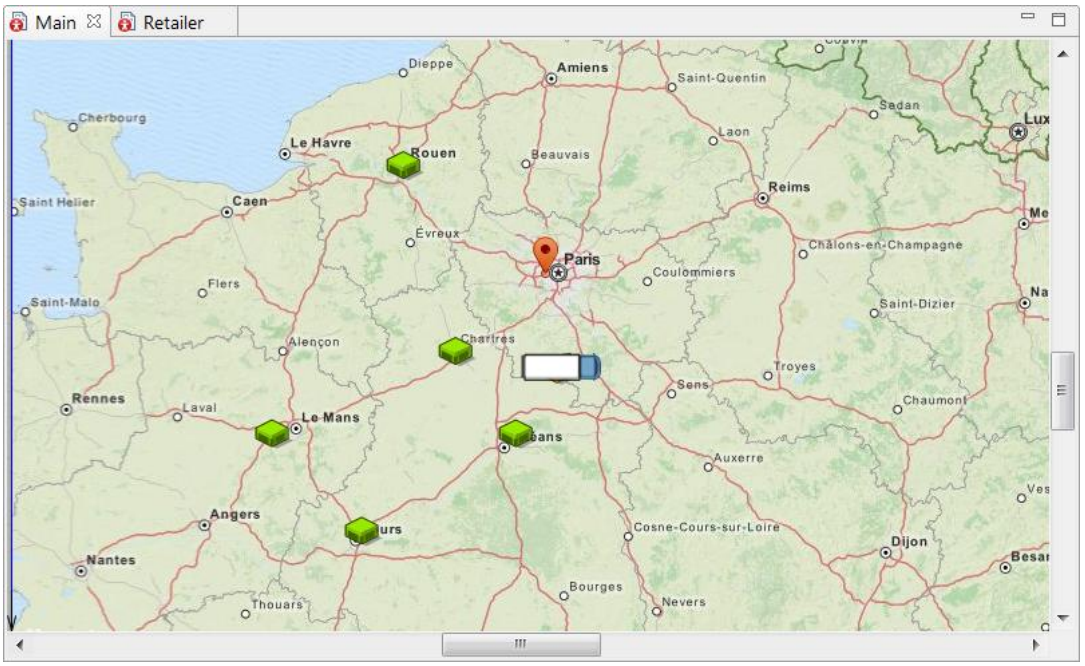
    And vice versa, when you select a temporary element on the map, the search result is highlighted in the **GIS Search Results** view.

11. Right-click the search result in the view and select the option **Create agents at selected elements** > **Retailer** from the context menu, or highlight an element and click the button  on the toolbar of the view.



12. The agent of this type will appear on the *Main* diagram, and its animation will be placed on the map in the specified location.

13. Repeat these steps to create several retailers on the map in the geographic locations you prefer: several single agents of the type *Retailer*.



14. Click the toolbar button of the **GIS Search Results** view to clear the view and the map, then close the view.

    Let us run the model: we have marked the main points of our supply chain on the GIS map.

    Next, we will define the *Retailer* behaviour: a retailer should generate a demand for the product to be delivered. This is where the process starts.

Phase 4. Sending orders from the retailer

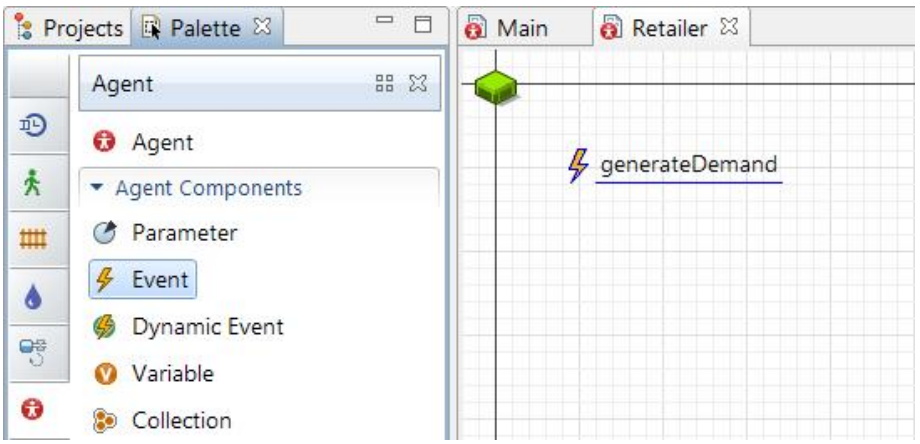
In this phase, we will model how any retailer can send a request to the distributor. We will tell a retailer how to form an order and then where to send it.

Retailers do not order the product delivery just once, and we want to model a supply chain that functions repeatedly.

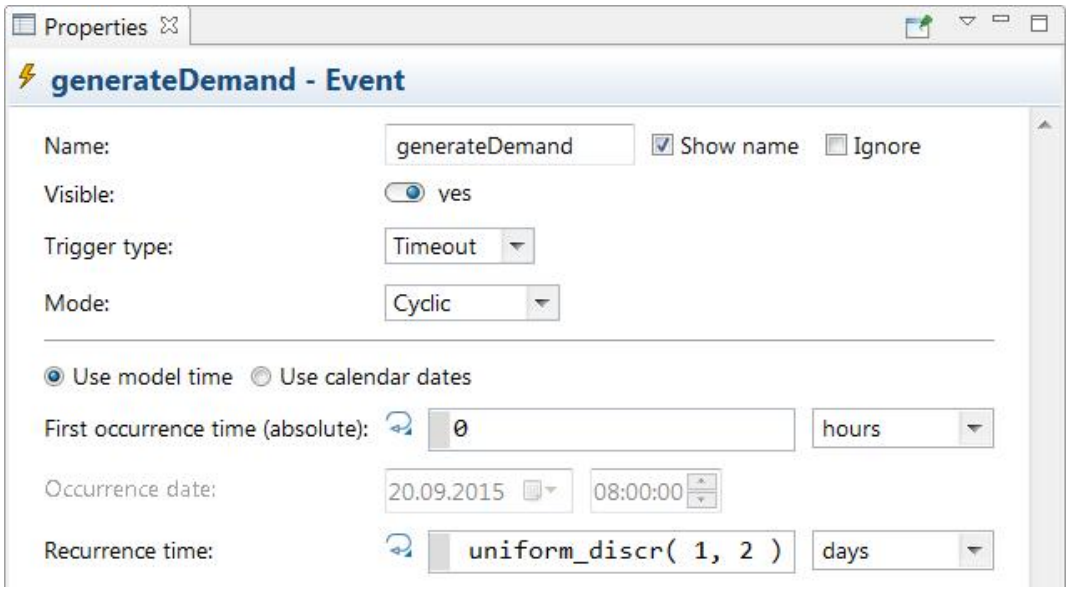
The best option to create a request like this is to use the **Event** object. The events in AnyLogic do not just happen, we can also assign to an event an action to perform.

1. In the **Projects** view, double-click the *Retailer* agent type to open its diagram. There you will find the animation figure we have selected for it in the **New agent** wizard.

2. Drag the **Event** object from the **Agent** palette. Type its name in the built-in editor: *generateDemand*.



3. For instance, we want our distributors to order more product every other day. We do not need to create a new event for each time it happens, we will use the special event mode called **Cyclic**. Then, since the mode is cyclic, we need to specify the **Recurrence time**.



Here, we use one of the standard distributions that AnyLogic provides, *uniform_discr(1, 2)*.

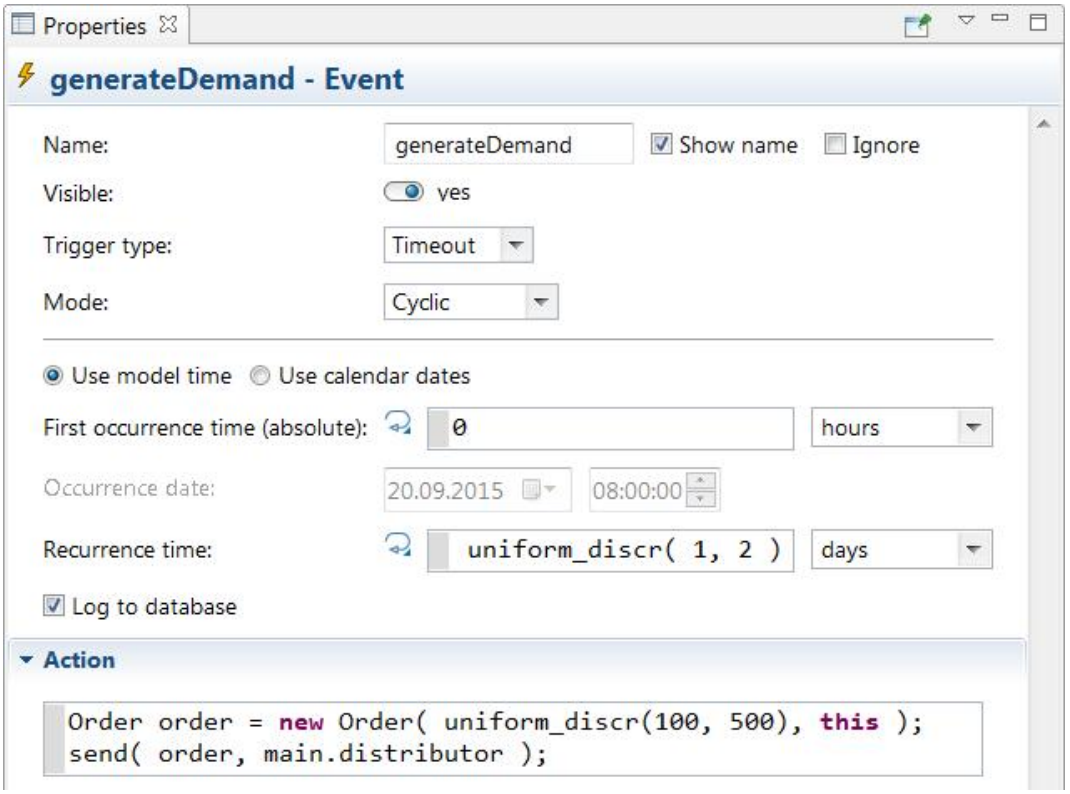
4. We have defined when the event should occur. Now let us tell AnyLogic what to do when the event occurs. Expand the **Action** properties section. It contains the edit box where we can type the code that will be called.

   First of all, we create an order of a special kind by calling the constructor of this type of agent. We use the parameters we have created before for the *Order* to define what kind of order it is.

   Then, we send the order that has been created to our distribution center:

   Order order = new Order(uniform_discr(100, 500), this);

   send(order, main.distributor);



5. We have set up the *Retailer* agent type logic.

   In the next phase, we will define how the distributor processes the order that it receives from a retailer.

Phase 5. Processing orders at the distributor

Let us continue defining the order processing in this phase, now on the distributor side.

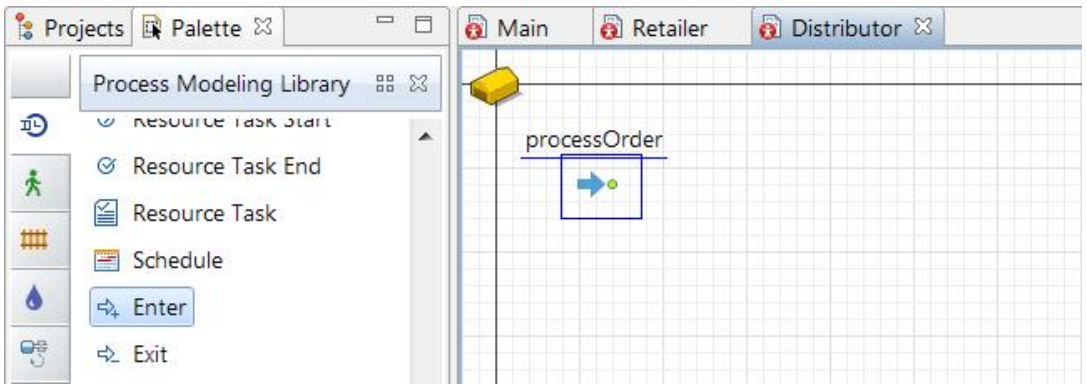
We will build a flowchart using the **Process Modeling Library** that manages the orders. Also, in this phase, we will finally create trucks (as you remember, the population on *Main* is still empty) and make them take part in the process as resources.

In the previous phase, we have set up an event that sends messages, now we will ensure the message delivery.

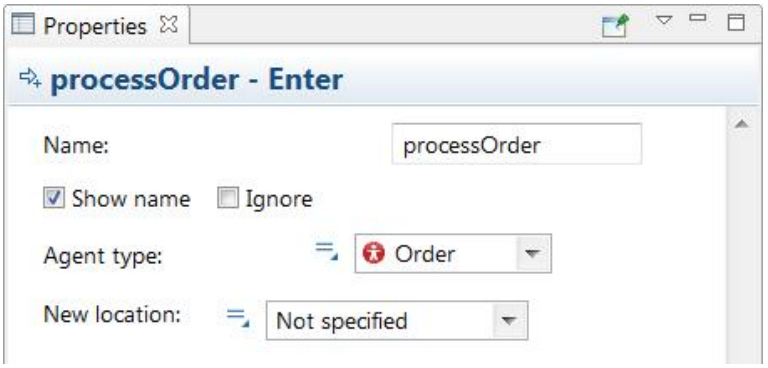
1. Go to the **Projects** view and double-click the *Distributor* agent type to open its diagram.

2. Switch to the **Palette** view and open the **Process Modeling Library**.

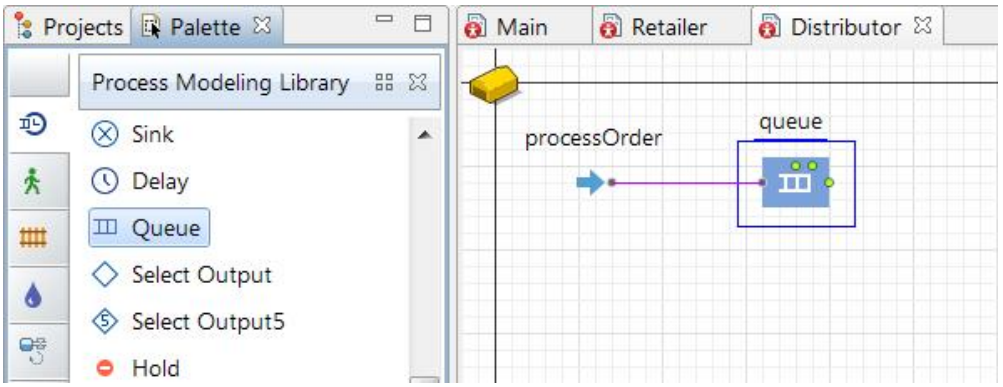
3. Start building the process flowchart with the library object **Enter**: drop it onto the diagram from the palette. This object inserts already existing agents in the process flowchart, and it is a common way to implement the process-centric modeling method in the GIS environment.



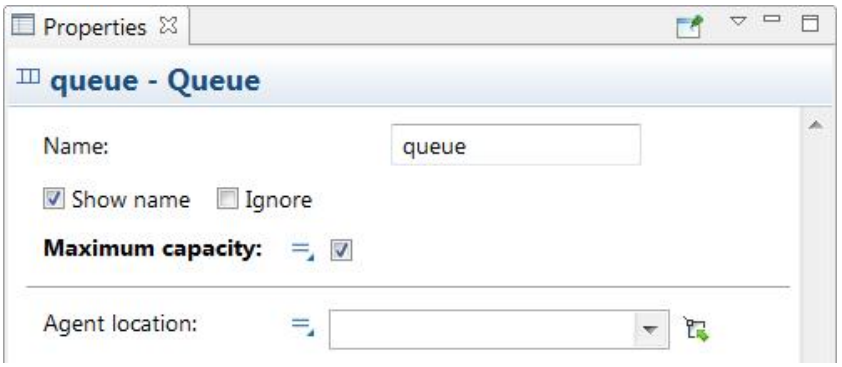
4. Name the object *processOrder*. We need to specify what kind of agent we want to insert in this flowchart: *Order*.



5. Next, drag the **Queue** object and let it connect with *processOrder* as shown in the figure below. When you place one library object close enough to another, they automatically connect, and you only need to follow the direction of the flowchart: from left to right.

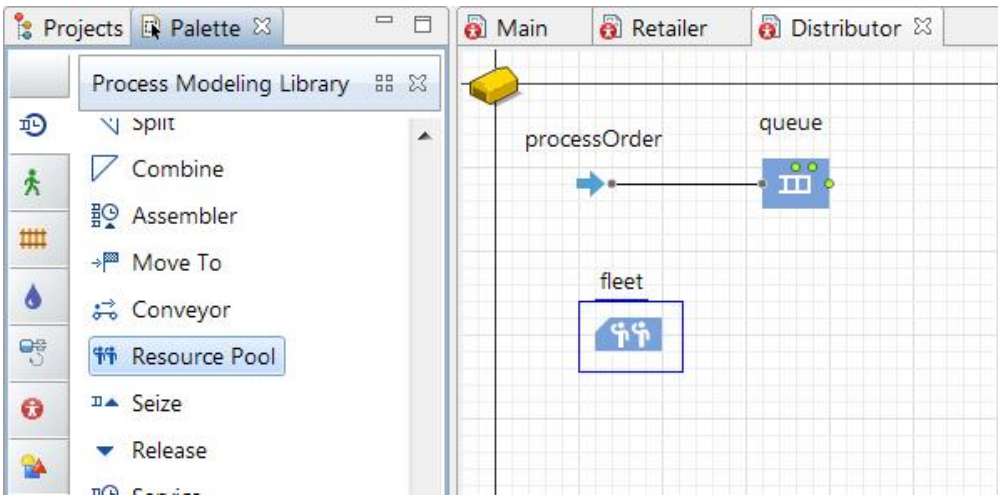


6. The orders will be first stored in the queue that allows **Maximum capacity**, we do not need any limits here in our model:



7. We will now continue building the flowchart. The second part will manage the resources: trucks in our model.

8. Let us now add the **ResourcePool** object that defines the resources' properties. This library object is a bit different from other objects because we do not need to connect it to the flowchart with connectors. We will select it in another object's property. Name the object *fleet* and go to its **Properties** view.

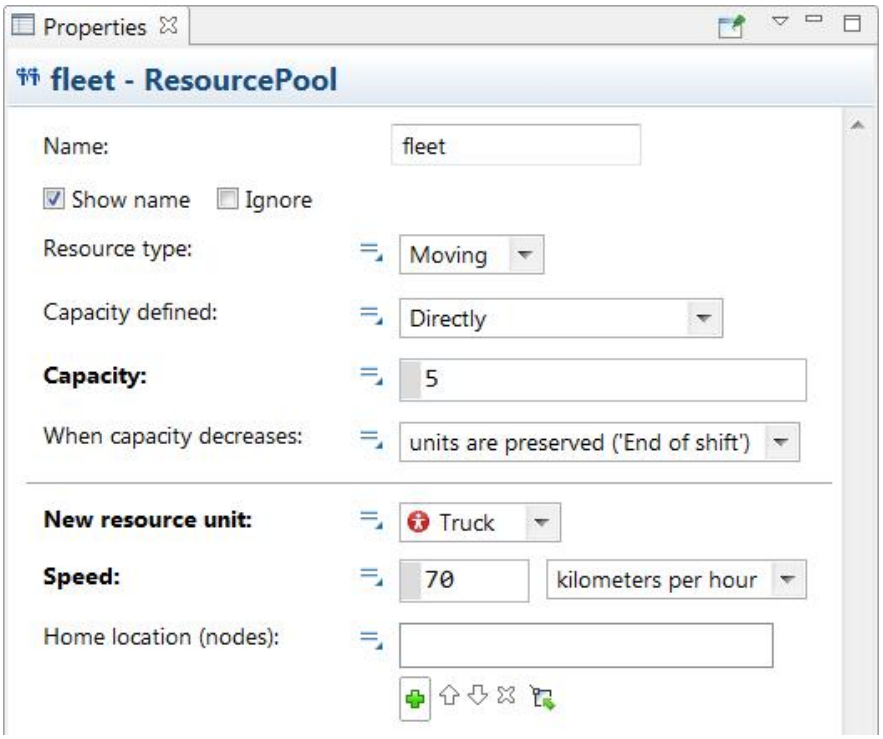


9. And now we will use this library object to create and define the trucks in our model. By default, the **Resource type** is set to *Moving* which suits us perfectly.

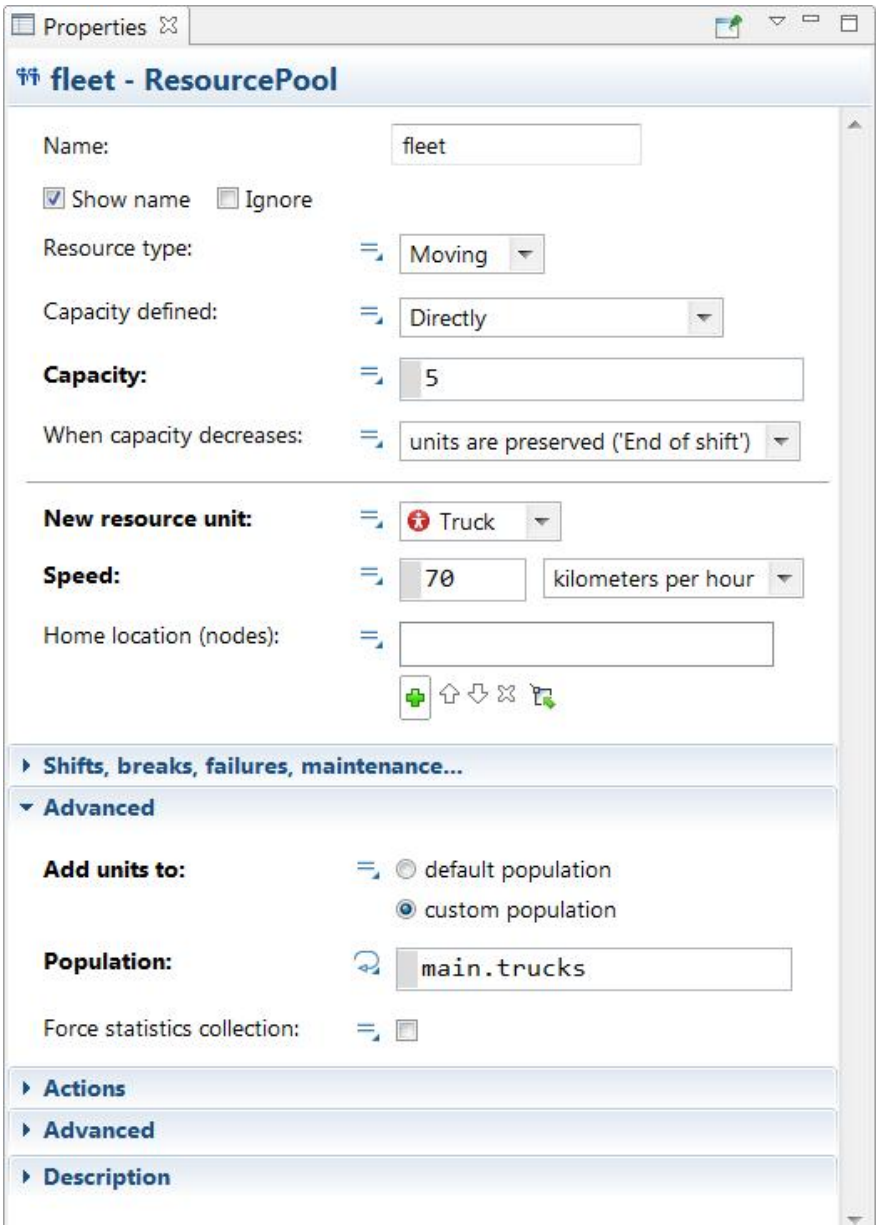
   Next, we can define the number of the trucks in our model by *directly* specifying the **Capacity** parameter. For this model, *5* trucks should be enough for our supply chain.

   None of the above will matter, if we do not tell the **Resource Pool** what kind of resources it should create and manage. Specify our agent *Truck* as the **New resource unit**, and then we will be able to use the trucks as resources in the flowchart.

   Here we can also specify the **Speed** parameter and assign a realistic speed value to the trucks in the desired units; for instance, *70 kilometers per hour*.

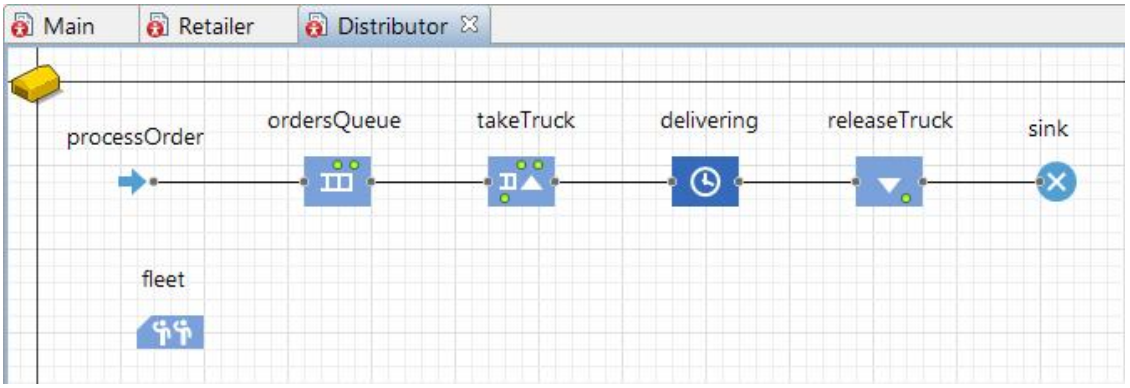


10. The last parameter of the trucks as resource units that we need to specify is their correct population. We want this **Resource Pool** to populate the *trucks[../]* agent. Expand the **Advanced** section of the fleet properties and choose to **Add units to**: *custom population*. Then, specify the original population on *Main* (*main.trucks*):

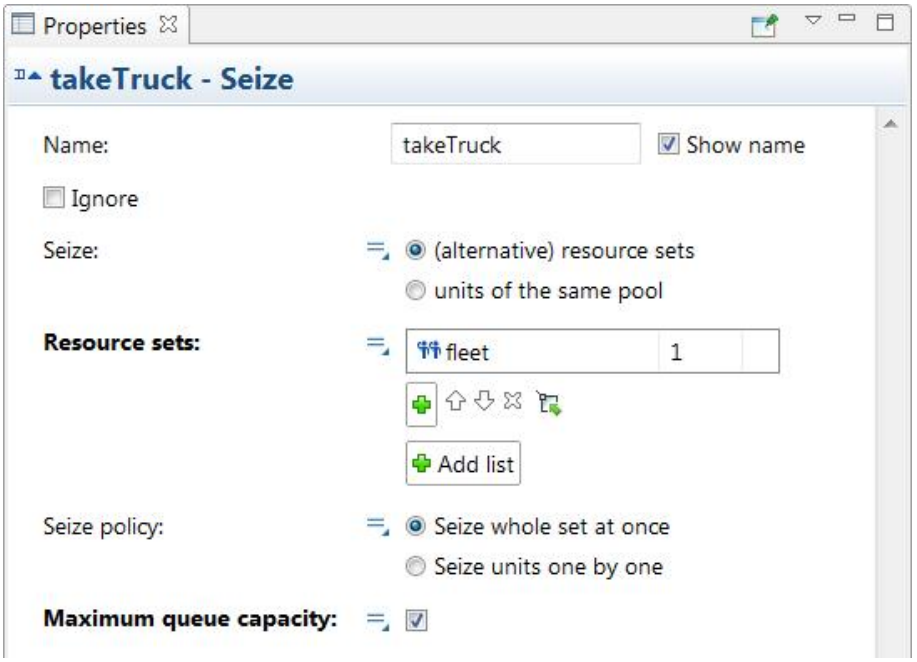


When AnyLogic suggests that you select some element from the drop-down list, but you need to type a piece of code in the parameter, click its icon to switch to the parameter's dynamic value mode.

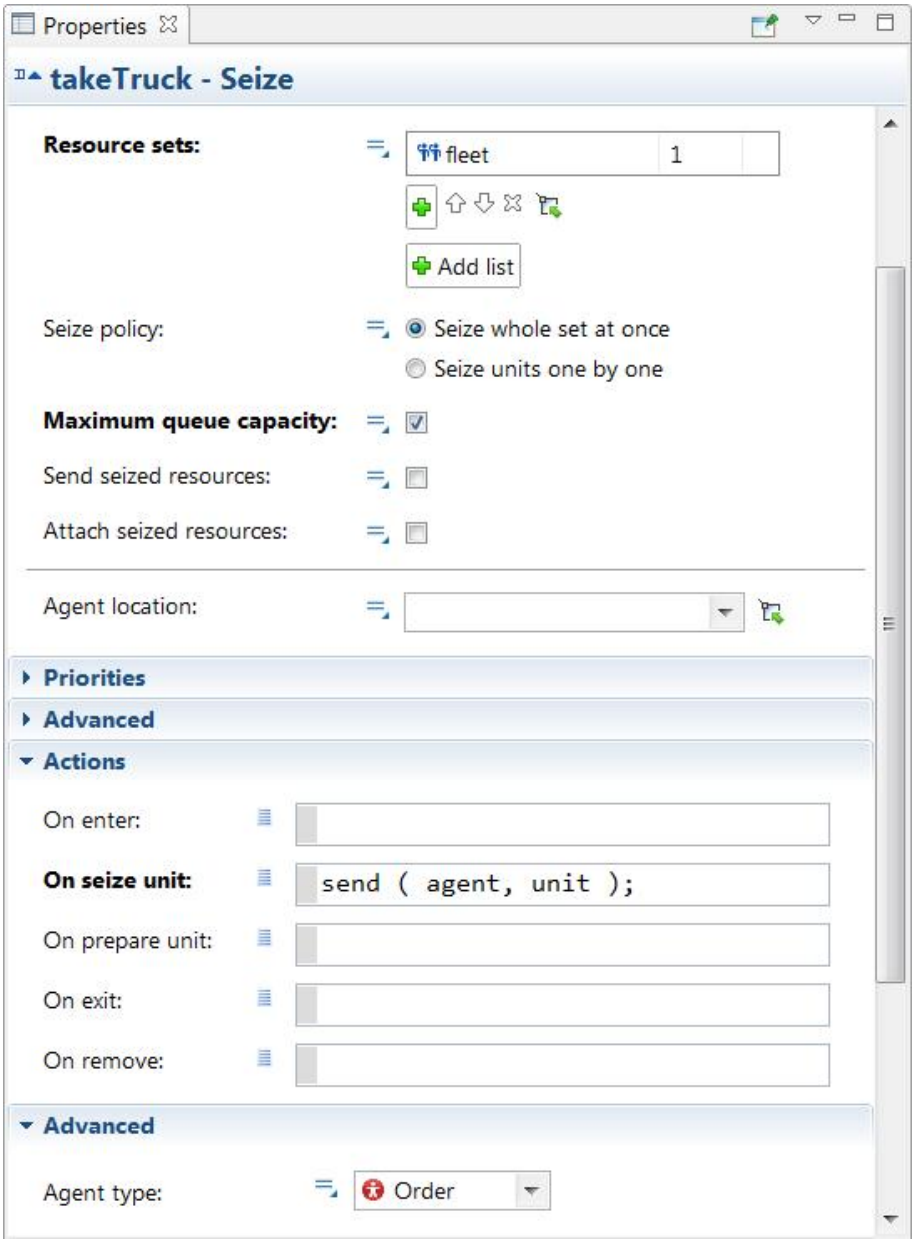
11. Let us continue building the process flowchart from where we have stopped. Name the queue *ordersQueue*. Add these four library objects, **Seize** – **Delay** – **Release** – **Sink**, and name them as shown in the figure (*takeTruck* – *delivering* – *releaseTruck* – *sink*):



12. The **Seize** object seizes the resources from the given **Resource Pool** object to utilize them. Select our *fleet* in the **Resource sets** property, then this object will use the resource units from this resource pool (our trucks). Enable the **Maximum queue capacity** - the queue where orders wait for an available truck.



13. Expand the **Actions** section, and enter the following code in the action **On seize unit**: *send (agent, unit);* When this object seizes a truck (*unit*), it sends an order (*agent*) to this truck, and the order, as you know, contains the information about the retailer that has sent it.

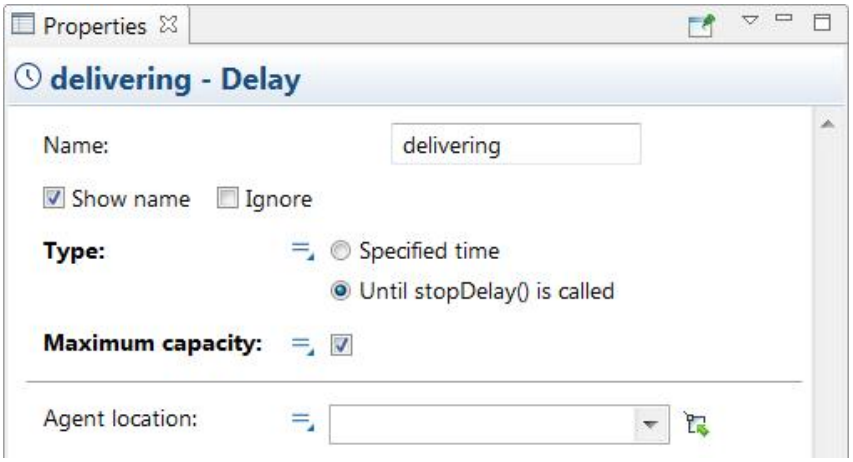


When you click in the edit box of any code parameter, you can see a small light bulb icon in its upper left corner. Hover over it to see the list of possible local variables.

Local variables here, *agent* and *unit*, allow you to access the **Agent type** and the **Resource type** correspondingly that we manage with this flowchart. We have selected the agent type (*Order*) in the *processOrder*, and it has propagated through the following flowchart blocks as the agent type whose process they define. The resource type (*Truck*) was set up in the **Resource Pool** that this **Seize** object uses.

14. The **Delay** object, that we called *delivering*, presents the time interval that it takes for a truck to deliver the product. Since our retailers are located in different places, we do not know exactly how long it is going to take to get to them. That is why the delay ends when the truck arrives and signals for it. We will model this action in the next phase.

    Now, we set the **Type** of delay to *Until stopDelay() is called*, and enable **Maximum capacity** to store orders without restrictions. That means, we have to call *stopDelay()* somewhere else.



15. The **Release** object naturally releases the resources that were taken by **Seize** when those units are no longer needed. Before the agents are disposed at the end of the flowchart, they have to release all of their resources. The **Sink** object disposes of agents and ends a flowchart.

16. Move the diagram down to find the connections object. This object is present on every agent's diagram; it manages the links to other agents in the model and supports communication between agents.

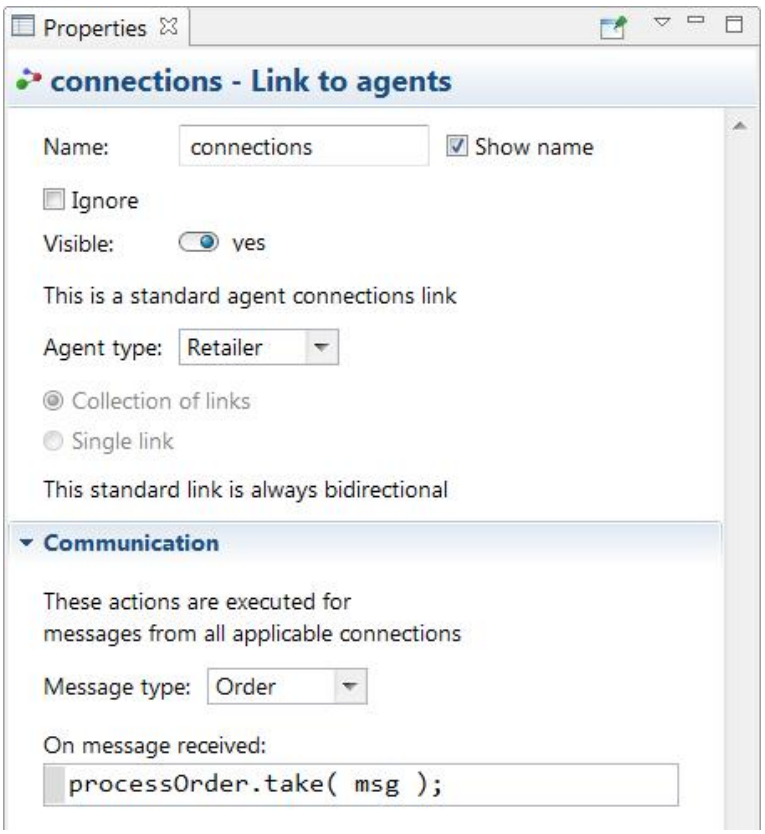
17. Communication between agents happens when they send and receive messages from each other. In the previous phase, we have created the event with the action that sends messages to the distributor. Now we will make sure the *Distributor* agent receives the messages and processes them correctly.

    The messages are sent from the *Retailer*, and that is why we need to choose it in the **Agent type** property. What we are sending is of the *Order* agent type, and we choose it as the **Message type** here.

    We have set up the message delivery of the special type we need. Next, let us tell *Distributor* what to do when it receives a message. Type the code line *processOrder.take(msg);* in the property **On message received**.

    take(agent) is a method that belongs to the **Enter** library object, which we named *processOrder*.

    When the message is received, the agent of type *Order* is inserted into the *processOrder* block, and the flowchart starts processing it, i.e., the distributor sends a truck to the retailer who demanded the product delivery.



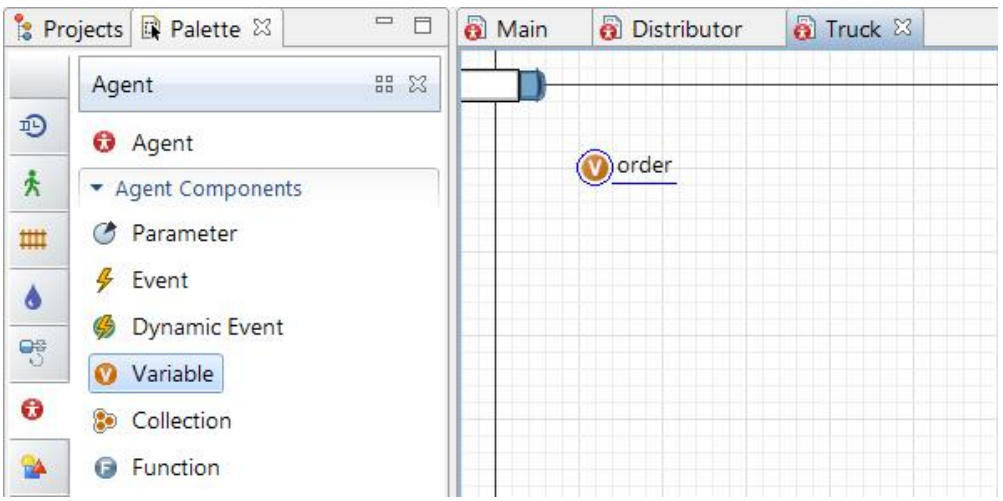
In the next phase, we will finally define the trucks' movement logic, from the distributor to one of the retailers and back.

Phase 6. Defining trucks' movement

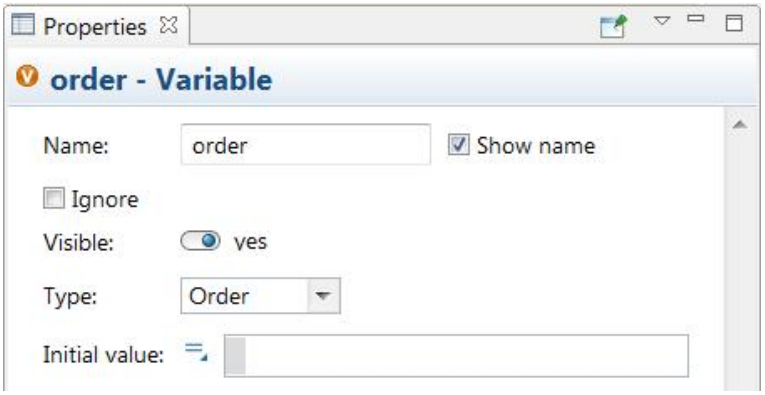
This is the last phase of our Supply Chain GIS model development process. We are ready now to define the trucks' movement between the points of the supply chain on the map.

We only need to specify the destination points for the trucks. All the routes are always available from the AnyLogic routing server, and by routes we mean real roads on the map.

1. Open the *Track* agent type diagram from the **Projects** view.
2. Drag the **Variable** object from the **Agent** palette. Name the variable *order*.

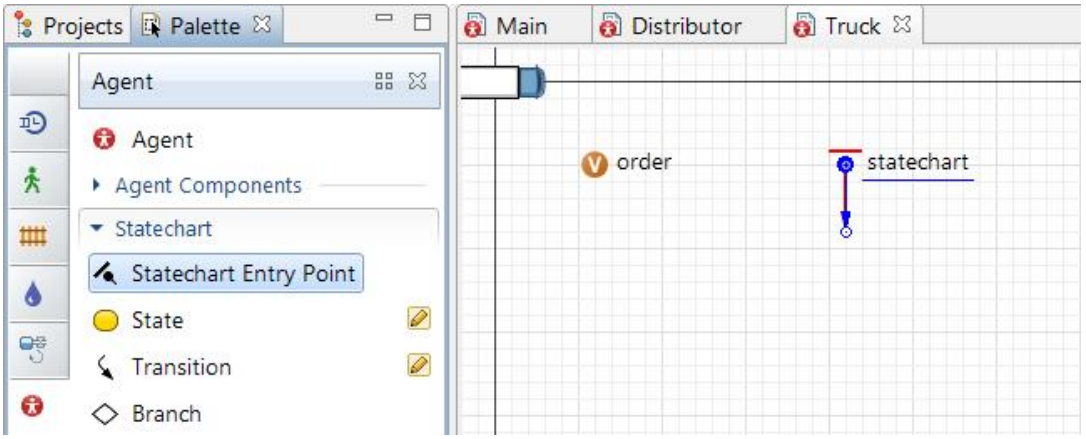


3. Choose its **Type**: *Order*. We will use this variable to access orders and the information they contain (as they are creating on *Retailer*) to enable trucks' movement in the statechart.

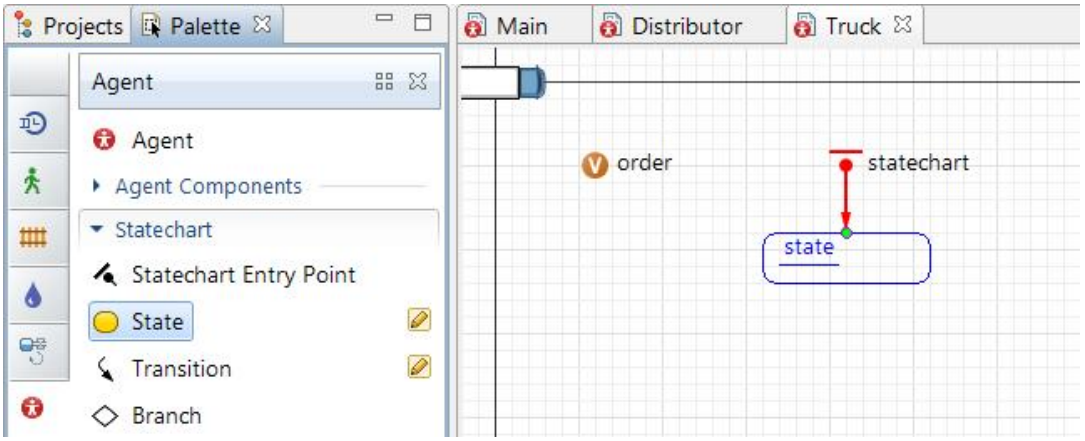


Any truck which received an order, should spend some time to load the product, then it moves to the retailer that requested this product, unloads the goods, and then it has to return back to the distributor. Each of these activities can be modeled as a state: truck is in the state of driving to a retailer. The transitions between the states switch a truck from one activity to the following action.

4. Open the **Statechart** palette in the **Palette** view. We will start building a statechart with the **Statechart Entry Point** object.



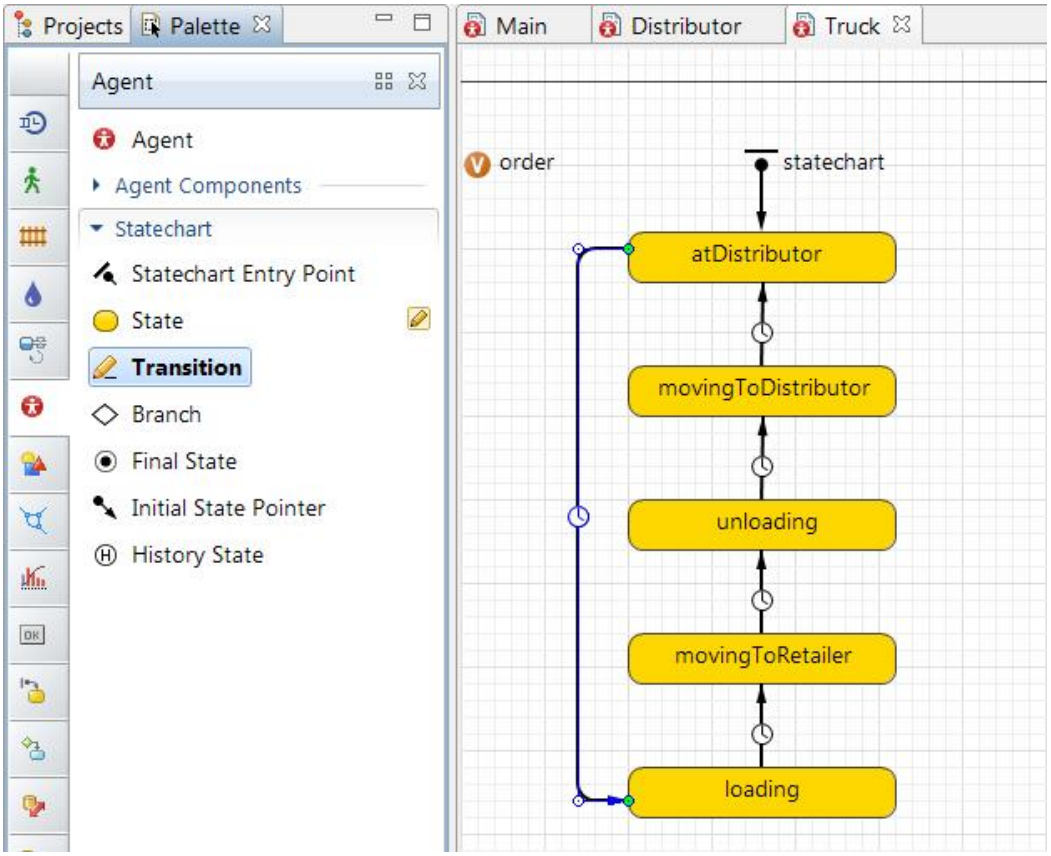
5. Next, drag from the palette the **State** object and connect it to the statechart entry point. When the elements are connected successfully, the point of connection is highlighted in the cyan color.



6. Building a statechart, we connect states with transitions. Add five states first on the *Truck* diagram and name them in the following way. Simply put them on the diagram one under another leaving some space for the transitions between them.



7. We will define the logic of the statechart in the properties of transitions. When we finish this statechart, it will look like in the figure below. You can see there are several types of transitions. Let us draw the transitions first, and then we will take a close look at each of them.



8. To draw a transition, double-click the **Transition** element in the **Palette** to activate its drawing mode. Then you can draw a transition by clicking first in the state that it goes from, then the state that it leads to.

   Be careful with the transitions' direction: one transition goes "down" from *atDistributor* to *loading*, and the transitions from *loading* to *atDistributor* go "up".

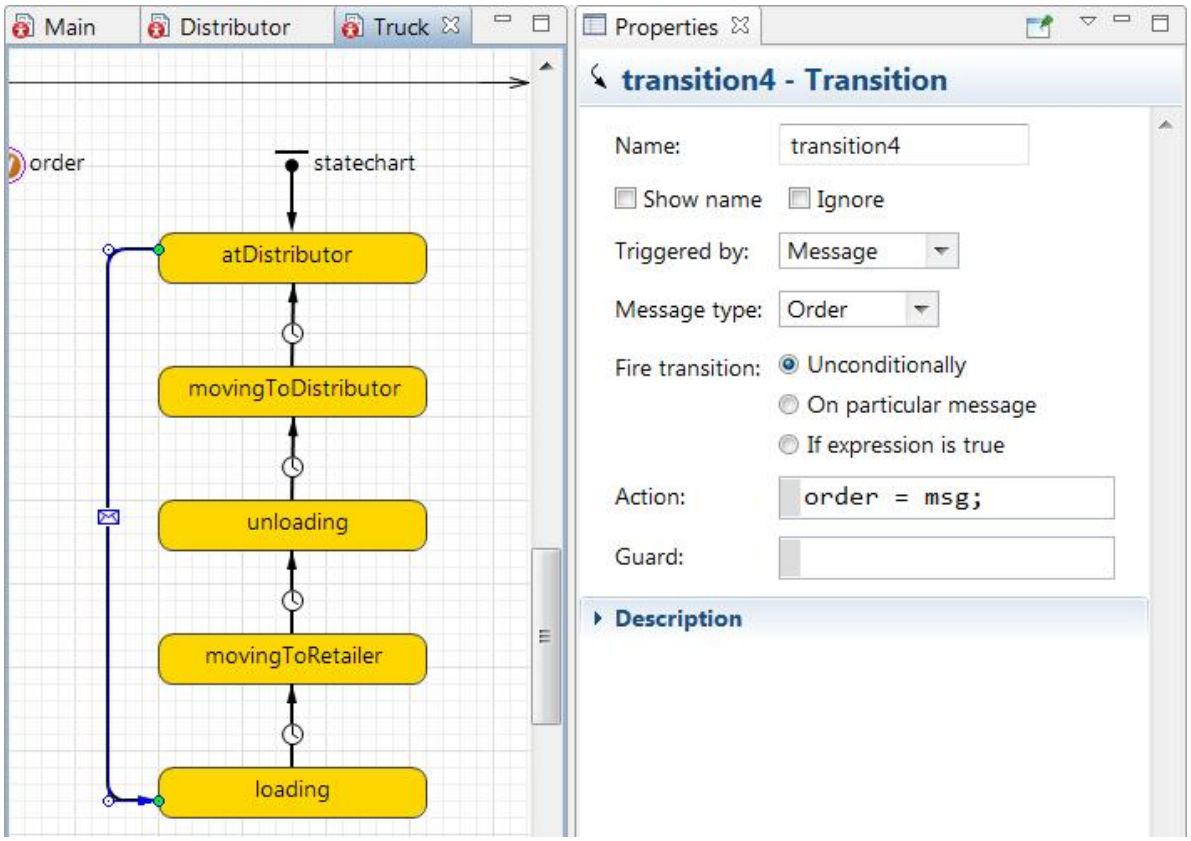
   By default, the trigger type of any transition is *Timeout*, and that is why it has an icon of a clock: 🕓. First you can draw "empty" timeout transitions, then we will define their properties, and change their appearance too.

9. Let us have a look at the transition from the state *atDistributor* to *loading*.

   atDistributor is the initial state in this statechart, we have set the *trucks[../]* initial location in the same GIS point as the distribution center. They are there when they receive an order.

   In the previous phase, we have defined the *takeTruck* object to send an *order* message to the trucks. This is where we set up the trucks to receive that message.

   We change the transition's trigger type to *Message* ✉, our message is of *Order* type. In the **Action** edit box, we assign the *order* variable the value of the local variable *msg* (*order = msg;*):

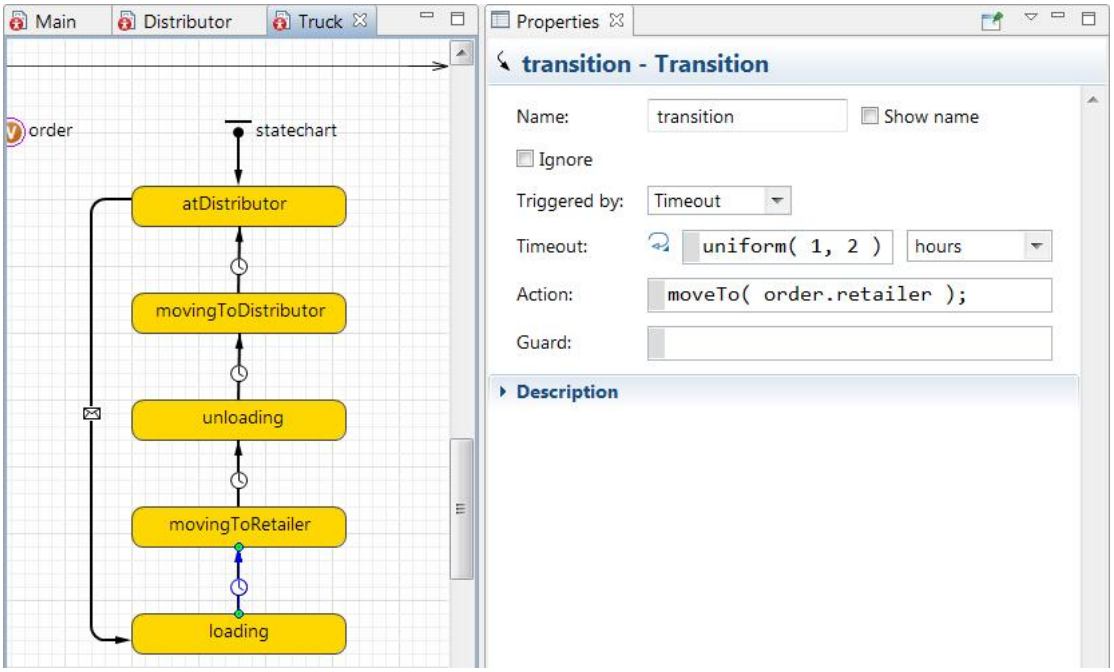


10. Now a truck is in the *loading* state. Loading takes time, so we will use the *Timeout* 🕓 trigger type for this transition.

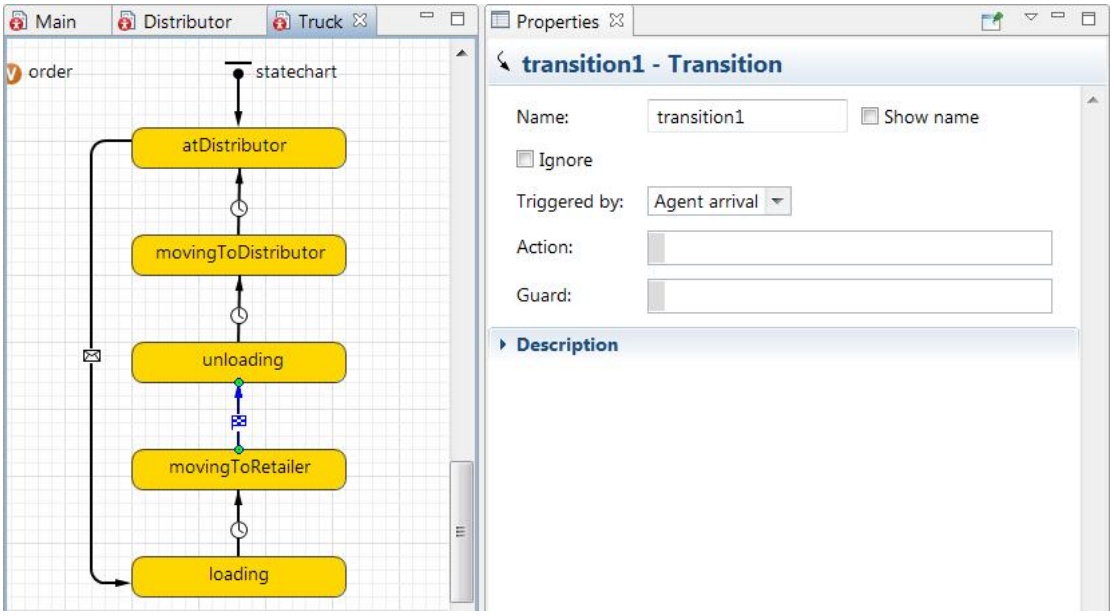
    Set the **Timeout** to *uniform(1, 2) hours*. Our next state is *movingToRetailer*, and we will switch a loading truck into this state after the specified timeout with the little piece of code in the **Action** edit box: *moveTo(order.retailer);*

    moveTo( ) is a popular function to define the agent movement. You can make an agent move to another agent, a node, even a geographic place. Here, we do not know which

retailer exactly may order the product this or the next time, so we set it to be that particular retailer that has sent the order that triggered the whole statechart in the first transition.

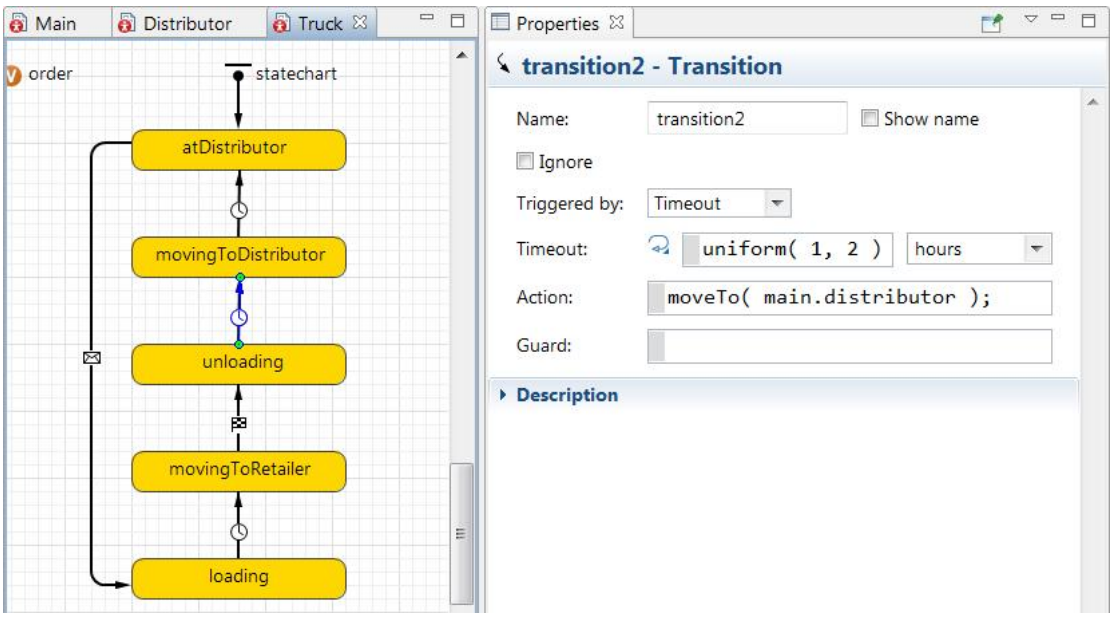


11. The *unloading* state should start as soon as the truck reaches its point of destination. The transition to *unloading* is **Triggered by**: *Agent arrival*.



12. Unloading itself takes some time, we trigger the transition from *unloading* to *movingToDistributor* by *Timeout*. Let it again be *uniform(1, 2) hours*. And after this timeout, we want the truck to go back to its base, the distributor.

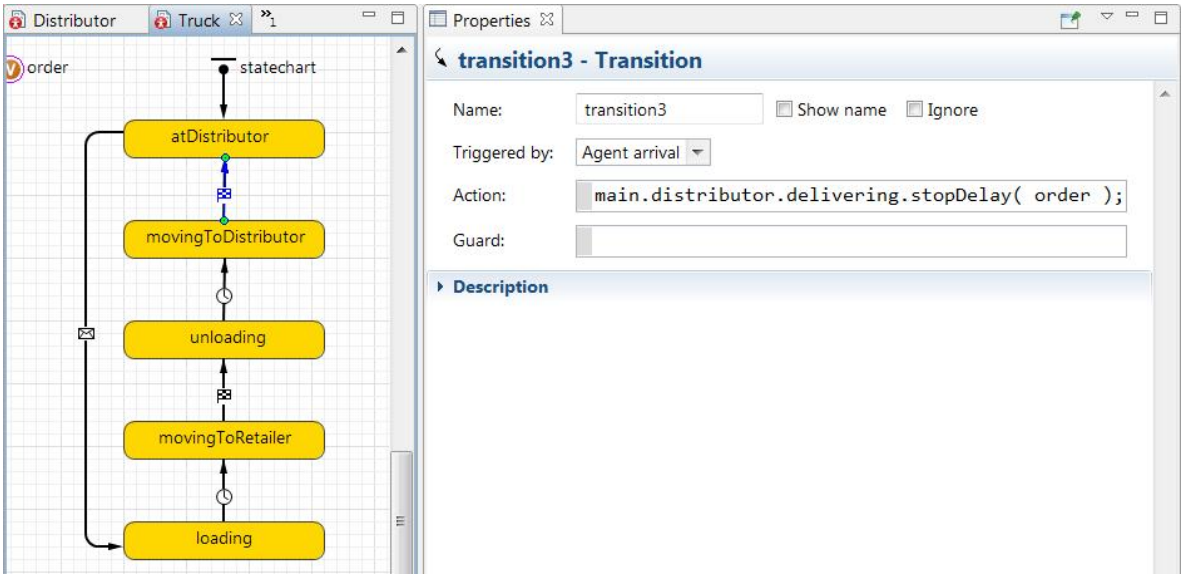
    The **Action** defines this movement: *moveTo(main.distributor);*



13. Once the truck arrives to its distributor, it is back in the *atDistributor* state, ready to take and process another order. This transition is triggered by *Agent arrival*.

    The **Action** here controls the flowchart on the *Distributor* diagram. The delay time lasted as long as a truck needed to deliver the product to the retailer, and now it is time to finish delay and release the resources:

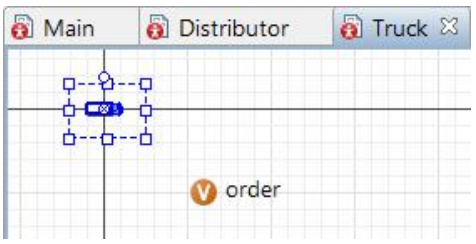
    main.distributor.delivering.stopDelay(order);



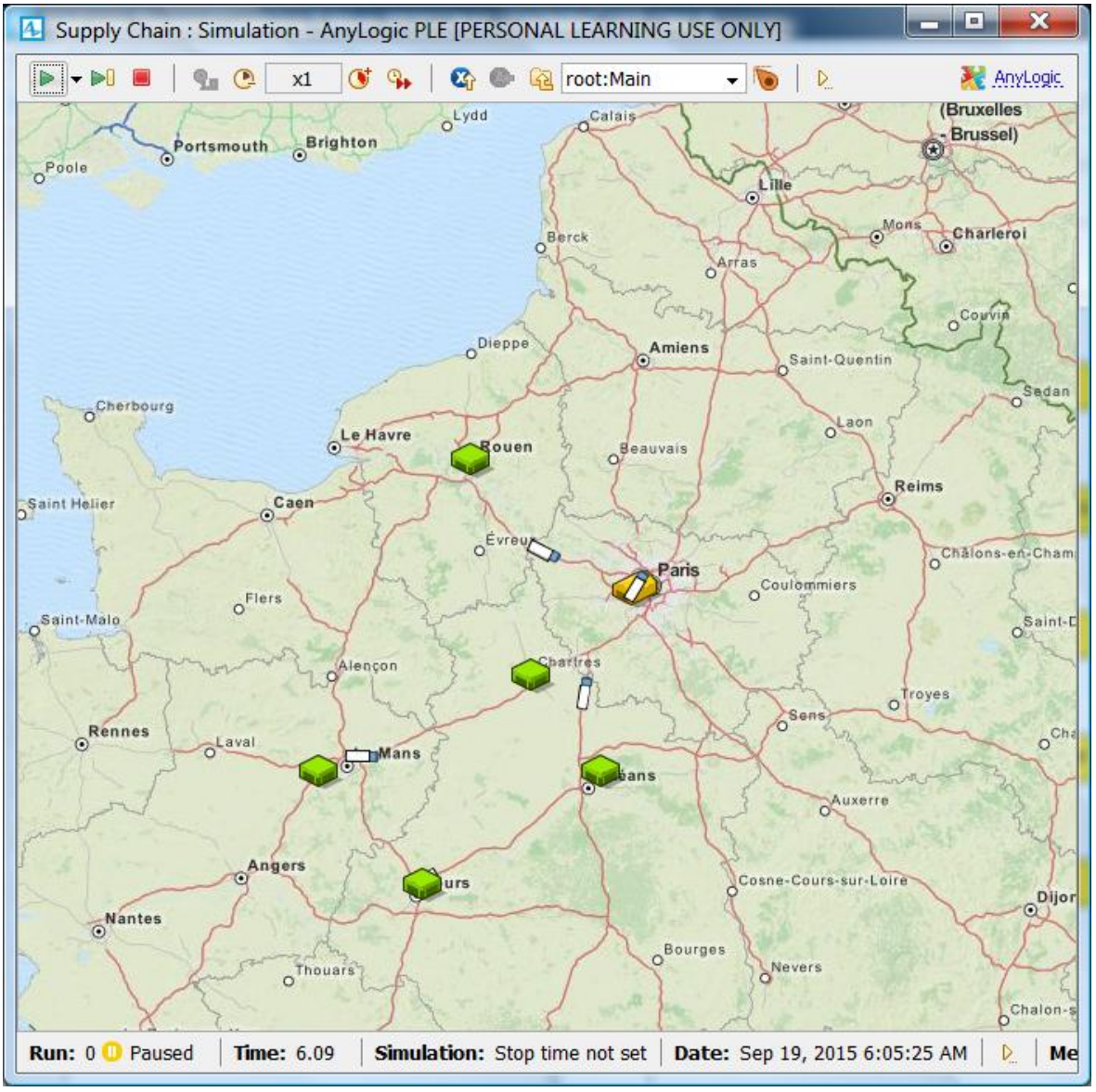
    stopDelay(agent) is a function of the **Delay** library object. We call this function for the argument *order* which is our variable on the *Truck* diagram. Its type is *Order*, the same agent type that the flowchart processes.

14. Now we are back to the first state. We have finished creating the statechart for the trucks.

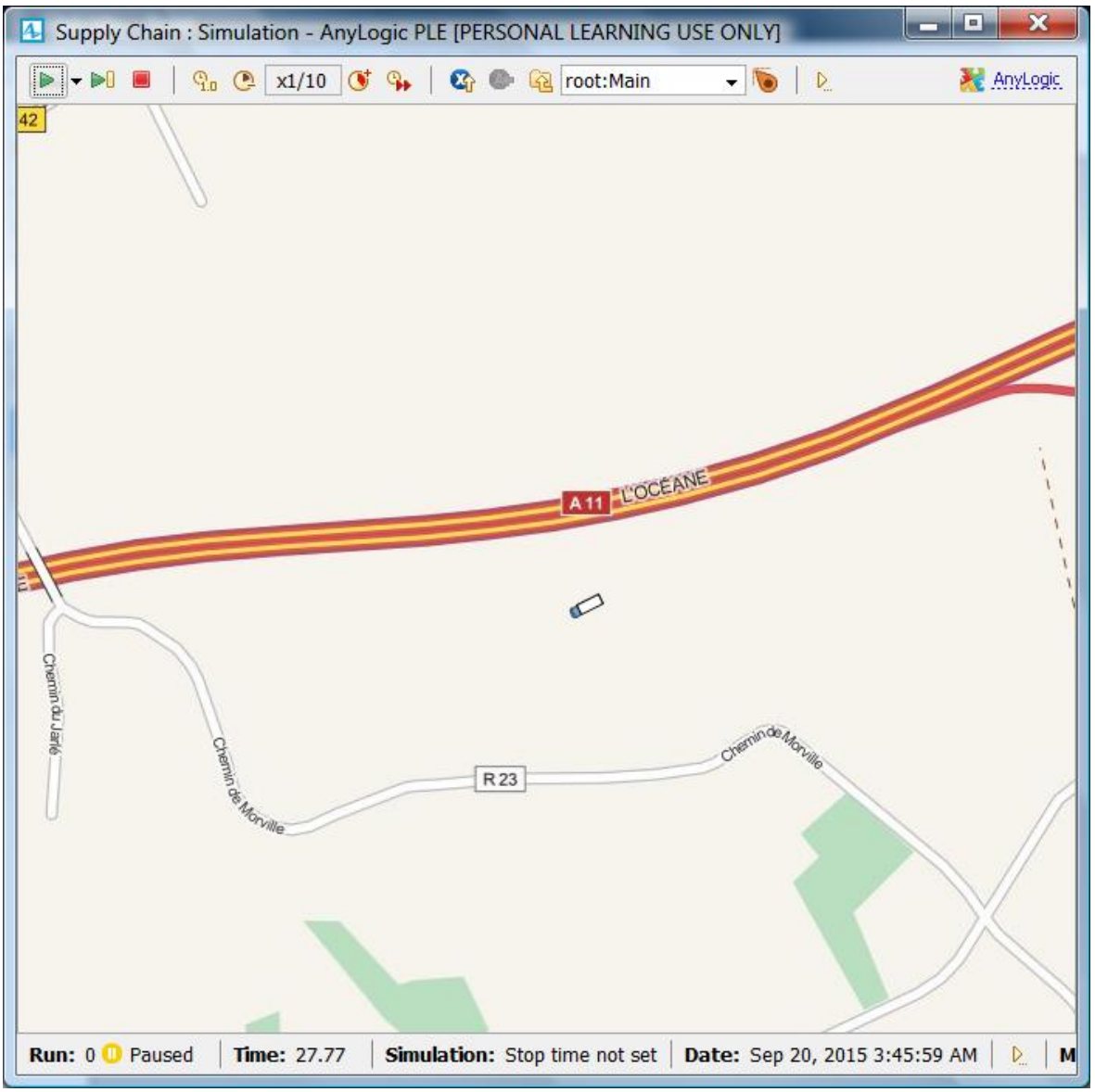
    As the final step, let us make the animation figure of a truck smaller. There are advanced scaling properties for the animation figures of agents on *Main* designed specially for the GIS map animation management.



15. This is the final stage, and we can run the model and observe the trucks going back and forth delivering product according to the requests that the center receives.

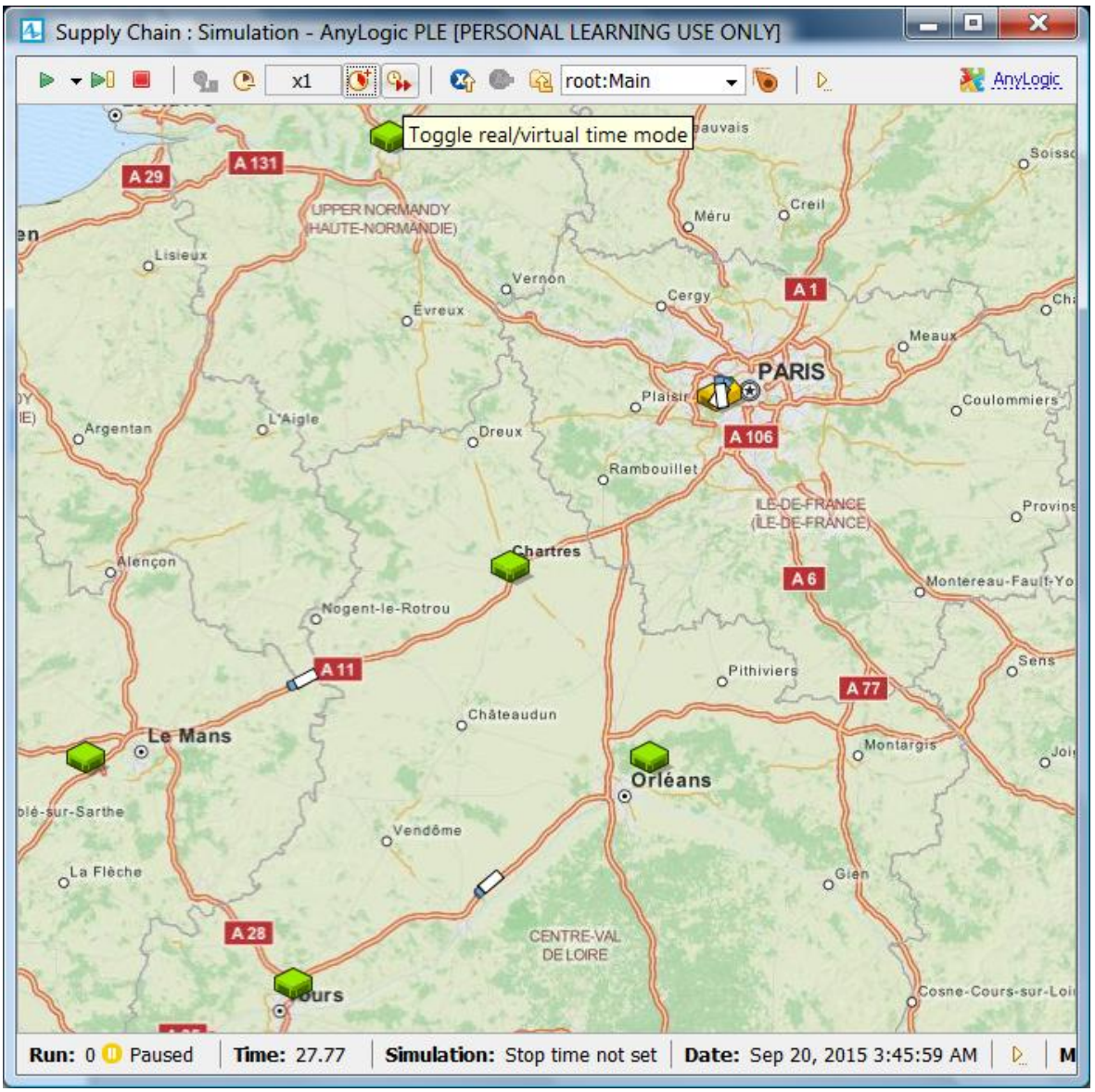


16. If you zoom in to a smaller scale, you will see how the trucks follow real roads on the map:



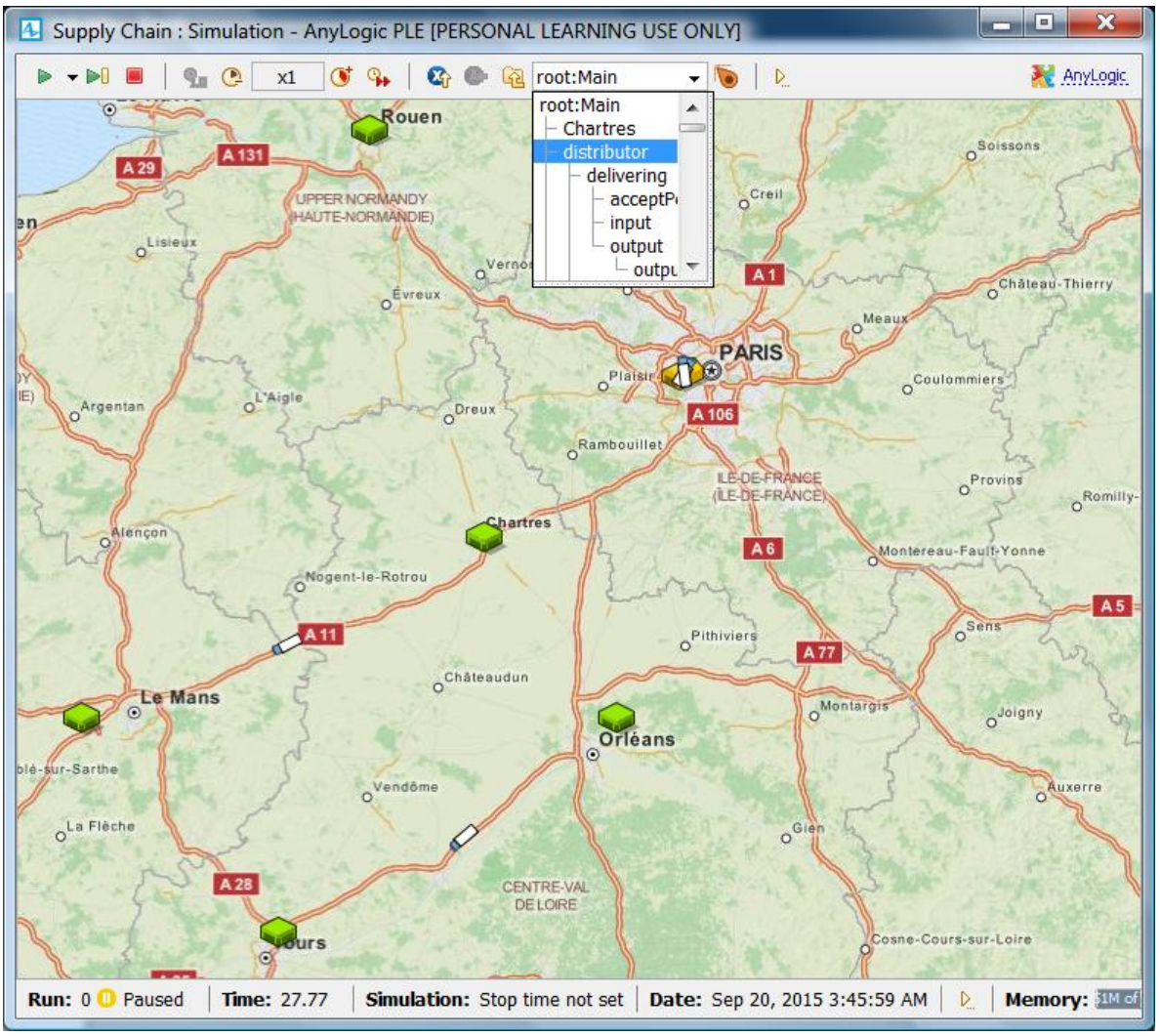
AnyLogic generalizes animation at the run-time for the sake of performance, although it always builds the routes with the best precision, both at the design-time and at runtime of the model, to calculate distances and travel time in compliance with the actual data. You can customize the generalization settings of the routes to make animation look even more realistic.

17. You can run the model in the **virtual time mode**, i.e., as fast as possible, to cache the routes quicker. After the routes are cached, the model performance will be higher.



To go back to the real time mode, click the button on the presentation window toolbar.

18. You can explore the agents in this model and observe their components. Click in the model explorer and select the agent type you would like to check.



19. For instance, we can take a look at the *Distributor* to check on the trucks utilization via *fleet* statistics. To go back to the *Main* presentation, click the "level up" button.